

CENTRAL ELECTRONICS LIMITED (CEL)

A Government of India Enterprise

CEL WEBSITE REDESIGN WITH OTP AUTHENTICATION AND COMPLAINT MANAGEMENT SYSTEM

INTERNSHIP PROJECT REPORT

Submitted by: Sarthak Verma

Designation: Intern

Internship Organization: Central Electronics Limited (CEL), Ghaziabad

Submitted To: Uma Ma'am

Department / Project Domain: Information Technology / Web Application Development

Submission Date: July 2026

Professional engineering report prepared from the submitted source code, configuration, dependency lockfile and project documentation.

Certificate

This is to certify that Mr. Sarthak Verma, working as an Intern at Central Electronics Limited (CEL), Ghaziabad, has prepared the internship project titled “CEL Website Redesign with OTP Authentication and Complaint Management System”. The work documented in this report analyses and explains the submitted implementation, including its Node.js and Express backend, SQLite database design, email-based OTP verification, password-reset workflow, role-based access controls, complaint routing, vendor resolution process, consumer confirmation process, comments, attachments and administrative statistics.

The report has been prepared for submission to Uma Ma’am. To the best of the available evidence, the technical descriptions, database tables, API routes, code explanations, diagrams and test results in this report are traceable to the project files submitted for assessment. The frontend directory and original user-interface screenshots were not included in the received upload; therefore, no unverified user-interface images have been fabricated.

Uma Ma’am
Project Guide / Reporting Officer

Head of Department / Authorized Signatory

Acknowledgement

I express my sincere gratitude to Central Electronics Limited (CEL), Ghaziabad, for providing the opportunity to undertake this engineering internship project. The project offered practical exposure to web application architecture, authentication, database design, complaint-management workflows, role-based authorization, file handling, email integration and production-readiness considerations.

I am especially thankful to Uma Ma'am for her guidance, review and support during the internship. Her direction helped align the work with the expectations of a professional public-sector organization and encouraged a structured approach to software design, documentation and testing.

I also acknowledge the value of the technical ecosystem used in the project, including Node.js, Express, SQLite, bcrypt, Nodemailer, Multer, Helmet and supporting open-source libraries. Working with these components strengthened my understanding of secure backend development and the importance of evidence-based engineering documentation.

Finally, I thank all officials, team members and well-wishers who contributed directly or indirectly to the completion of the internship and this report.

Sarthak Verma

Abstract

The submitted project is an internal prototype for a redesigned Central Electronics Limited website combined with an email-verified complaint management platform. The verified implementation uses a single Node.js and Express application, a SQLite database accessed through better-sqlite3, bcrypt password hashing, six-digit email OTP verification, password-reset codes, session-based authentication, role checks, attachment uploads, complaint comments and status email notifications. The code supports three operational personas: consumer, vendor and administrator.

A consumer or vendor registers under one of four manufacturing categories: Defence & Strategic Systems, Industrial Electronics & IT, Railway Technologies or Renewable Energy. Product Issue complaints inherit the complainant's registered manufacturing category, preventing arbitrary client-side routing. An administrator assigns such a complaint to a verified vendor in the same category. The vendor records a resolution, after which the consumer either confirms closure or disputes the resolution. Direct complaints in HR, Payment or General remain under administrative control. Complaint states implemented in the source are Open, Assigned, Vendor Resolved, Disputed and Solved.

The report was produced by examining every uploaded artifact: environment configuration, example configuration, package manifest, lockfile, README and 665-line server source. The JavaScript source passed syntax validation. The SQL schema was extracted from the application, instantiated in SQLite, and tested for unique, CHECK and cascading foreign-key constraints. Runtime installation was attempted but was blocked by HTTP 503 responses from the package registry, while the documented public frontend directory was absent from the upload. Accordingly, the report includes actual source-code screenshots, actual generated schema screenshots, verified test output and clear capture instructions for the missing user-interface evidence rather than invented screens.

The project demonstrates a coherent prototype architecture and a meaningful complaint lifecycle. The principal production gaps identified are the absence of the submitted frontend, non-persistent default session storage despite a documented SQLite-backed session claim, public static serving of uploaded files, disabled Content Security Policy, local-file storage, mock OTP exposure in development mode and the need for stronger audit, CSRF, malware-scanning and deployment controls.

Table 1: Document evidence inventory

<u>Artifact</u>	<u>Purpose</u>	<u>Analysis status</u>
.env	Runtime configuration keys; secret-like values redacted	Supplied and inspected
.env.example	Documented development configuration interface	Supplied and inspected
package.json	Scripts and declared dependencies	Supplied and parsed
package-lock.json	Exact dependency graph and locked versions	Supplied and parsed
README.md	Features, intended structure and run instructions	Supplied and inspected
server.js	665-line backend, schema and API implementation	Supplied; syntax validated
public/ and screenshots	Frontend and original UI evidence	Not supplied

Table of Contents

Section	Page
Certificate	2
Acknowledgement	3
Abstract	4
Table of Contents	6
List of Figures	8
List of Tables	10
1. Introduction	12
2. Organization Profile	14
3. Project Overview	15
4. Technologies Used	16
5. System Requirements	18
6. System Architecture	19
7. Database Design	23
8. Module Description	28
9. Source Code Explanation	30
10. Project Folder Structure	32
11. User Interface	34
12. Coding Screenshots	36
13. Database Screenshots	50
14. Working Process	53
15. Testing	54
16. Results	57
17. Challenges Faced	58
18. Future Enhancements	59
19. Conclusion	60
20. References	61
Appendix A. API Catalogue	62

<u>Section</u>	<u>Page</u>
Appendix B. Evidence and Traceability Matrix	63

Word compatibility — The document uses structured Heading 1–3 styles. Microsoft Word can generate or refresh an automatic table of contents from these headings if institutional formatting requires live fields.

List of Figures

Figure	Caption	Page
Figure 1	High-level system architecture	19
Figure 2	End-to-end complaint workflow	20
Figure 3	Level-0 data flow diagram	21
Figure 4	Functional module diagram	22
Figure 5	Deployment view and recommended production boundary	22
Figure 6	Entity-relationship diagram	23
Figure 7	Complaint state-transition model	27
Figure 8	Uploaded artifact inventory and verified omissions	32
Figure 9	Documented intended project hierarchy	32
Figure 10	Application bootstrap and domain constants	37
Figure 11	SQLite tables, constraints and indexes	38
Figure 12	Admin seeding, session configuration, security middleware and uploads	39
Figure 13	Validation, role guards and email services	40
Figure 14	Registration, OTP verification and login	41
Figure 15	Password reset and complaint query model	42
Figure 16	Complaint listing, summary and creation	43
Figure 17	Complaint detail, comments and admin mediation	44
Figure 18	Vendor resolution, consumer confirmation and admin statistics	45
Figure 19	Declared package dependencies and scripts	46
Figure 20	Documented functional and security features	47
Figure 21	Documented hierarchy and execution instructions	48
Figure 22	Environment-variable interface using development example values	49
Figure 23	Core SQLite table definitions and relationships	50
Figure 24	Supporting SQLite tables for OTP, reset and comments	51
Figure 25	Generated SQLite schema object inventory	52
Figure 26	JavaScript syntax check and runtime tool versions	55
Figure 27	Static validation and SQLite constraint test output	55
Figure 28	Dependency installation limitation in the analysis environment	56

List of Tables

<u>Table</u>	<u>Title</u>	<u>Page</u>
Table 1	Document evidence inventory	4
Table 2	Project objectives	12
Table 3	Benefits by stakeholder	13
Table 4	Technology stack and verified versions	16
Table 5	Hardware requirements	18
Table 6	Software requirements	18
Table 7	Architecture component responsibilities	19
Table 8	Database relationships	23
Table 9	users table data dictionary	24
Table 10	email_otps table data dictionary	24
Table 11	password_resets table data dictionary	24
Table 12	complaints table data dictionary	26
Table 13	comments table data dictionary	27
Table 14	Module input-output summary	28
Table 15	Important functions and responsibilities	30
Table 16	Source-code traceability by line range	30
Table 17	Uploaded versus documented folder structure	32
Table 18	UI screenshot evidence status	34
Table 19	Coding screenshot index	36
Table 20	Database screenshot index	52
Table 21	Test cases and results	54
Table 22	Verified achievements and limitations	57
Table 23	Challenges and resolutions	58
Table 24	Future enhancement roadmap	59
Table 25	Reference sources	61
Table 26	REST API catalogue	62
Table 27	Evidence and traceability matrix	63

1. Introduction

1.1 Project Background

Public-sector organizations increasingly require websites that serve two roles: an authoritative public information channel and a secure operational interface for citizens, customers, vendors and internal officials. The submitted project addresses both roles by combining a modernized CEL web presence with a complaint-management workflow. The README describes a public website with accessibility controls, responsive navigation, business-area content, news, tender previews and products, while server.js implements the authentication and complaint APIs that support the user, vendor and administrator portals.

1.2 Problem Statement

The project addresses the need for a unified, traceable and role-controlled system through which users can create accounts, verify ownership of their email addresses, register complaints, attach supporting documents, exchange comments and follow complaint progress. Product-related cases must be routed to an appropriate vendor without allowing a user to manipulate the manufacturing category. Administrative staff require a consolidated view, assignment controls, status updates and statistics. Vendors require a restricted queue containing only complaints assigned to them, and consumers must retain the final right to accept or dispute a vendor resolution.

1.3 Objectives

Table 2: Project objectives

ID	Objective	Implementation evidence
O1	Provide a modern CEL website and portal architecture	README public website feature set and Express static serving
O2	Implement secure account registration and login	bcrypt hashing, OTP verification, sessions and rate limits
O3	Support consumer, vendor and administrator personas	user_type and role columns plus middleware checks
O4	Route product complaints by manufacturing category	category inherited from the authenticated user profile
O5	Provide an auditable complaint lifecycle	status model, timestamps, remarks and comments
O6	Allow supporting attachments	Multer disk storage, extension whitelist and 5 MB limit
O7	Provide administrative reporting	pagination, filters, summaries and statistics endpoints
O8	Remain simple to demonstrate and hand over	single Node.js process and local SQLite database

1.4 Scope

- Public website shell and responsive frontend as documented in README.md.
- Account registration for consumers, vendors and staff administrators.
- Email OTP verification, resend and password-reset code workflows.
- Session login/logout and current-user discovery.
- Complaint creation, listing, filtering, pagination, details and comments.
- Product Issue routing through administrators to category-matched vendors.
- Vendor resolution, consumer confirmation or dispute, and status email notifications.
- Administrative vendor lookup and aggregate statistics.

1.5 Benefits

Table 3: Benefits by stakeholder

<u>Stakeholder</u>	<u>Benefit</u>
Consumers	Single complaint ID, status visibility, supporting attachments, comments and final closure control
Vendors	Restricted assigned queue and structured resolution remarks
Administrators	Centralized mediation, category-based assignment, official remarks and statistics
CEL	Traceable digital workflow, lower manual follow-up and a base for future institutional deployment
Developers	Compact stack, automatic schema creation, migrations and documented configuration

2. Organization Profile

Central Electronics Limited (CEL) is a Government of India enterprise under the Department of Scientific and Industrial Research (DSIR), Ministry of Science and Technology. CEL was established in 1974 to commercially exploit indigenous technologies developed by national laboratories and research institutions. Its corporate office and factory are located in Sahibabad, Ghaziabad, Uttar Pradesh.

CEL is identified on its official website as a Mini Ratna enterprise. Its current business areas include Renewable Energy, Railway Technologies, Integrated Engineering Solutions, Defence and Strategic Systems, Digital Infrastructure, and Research and Development. The official history highlights CEL's work in solar photovoltaic technology, railway axle-counter systems, strategic electronic components and integrated turnkey solutions.

The organization's published vision emphasizes trusted, technology-innovative public-sector delivery across defence, green energy, strategic electronics, railways and associated services. Its mission focuses on indigenous, quality solutions grounded in innovation, integrity and national priorities. These characteristics make a secure, accessible and well-governed digital platform directly relevant to CEL's public interface and operational support requirements.

Source note — Organization-profile facts are drawn from CEL's official About, Home, Contact and Vision pages accessed in July 2026. Technical claims elsewhere in the report are derived from the uploaded project artifacts.

3. Project Overview

3.1 Purpose

The project provides a combined public website redesign and complaint management prototype. The public-facing portion is intended to present CEL's identity, business areas, leadership message, news, quick links, tenders, products, partner logos and accessibility controls. The portal portion adds secure account and complaint functions through REST-style JSON APIs.

3.2 Verified Features

- Email-verified account registration using a six-digit OTP with a 10-minute expiry.
- Passwords hashed with bcrypt using cost factor 12.
- Consumer and vendor registration under one of four manufacturing categories.
- Optional administrator self-registration controlled by a secret staff code, plus automatic seeded administrator account.
- Password reset with a separate hashed six-digit code, expiry, attempt counter and generic anti-enumeration response.
- Complaint creation with automatic IDs in the format CEL-YYYY-000001.
- Direct complaint categories: HR, Payment and General.
- Product Issue category derived from the authenticated profile and assigned only to a vendor in the same category.
- Comment threads accessible only to the owner, assigned vendor or administrator.
- Attachment whitelist for PDF, PNG, JPG/JPEG, DOC and DOCX, limited to 5 MB.
- Status email notifications through mock console output or SMTP.
- Pagination and filters by category, status and subject search.
- Administrative statistics for total, solved, unsolved, user types, assignment queue and disputes.

3.3 Working Principle

The application starts by creating the data and uploads directories, opening data/cel.sqlite, enabling write-ahead logging and foreign keys, and executing idempotent CREATE TABLE statements. It also contains migrations for older users and complaints schemas and seeds an administrator if the configured administrator email is not present. Express middleware then configures security headers, JSON and form parsing, sessions, static assets, rate limits and Multer uploads. API routes execute parameterized SQLite statements and return JSON responses.

3.4 Advantages

- Low operational complexity for demonstration because no external database server is required.
- Strong domain separation among consumers, vendors and administrators.
- Database-level uniqueness, CHECK and foreign-key constraints supplement application validation.
- Complaint category is not trusted from the client for Product Issue cases.
- Automatic migrations preserve older complaint records as Direct/General cases.
- Mock email mode enables local development without SMTP credentials.

4. Technologies Used

The following stack is verified from package.json and package-lock.json. “Declared” is the semver range requested by the manifest; “Locked” is the concrete version recorded in the lockfile.

Table 4: Technology stack and verified versions

Technology / package	Declared	Locked	Purpose
bcryptjs	^2.4.3	2.4.3	Password hashing with cost factor 12
better-sqlite3	^11.7.0	11.10.0	Synchronous SQLite database access
connect-sqlite3	^0.9.15	0.9.17	Declared session-store dependency; not wired into server.js
dotenv	^16.4.7	16.6.1	Loads environment configuration
express	^4.21.2	4.22.2	HTTP server and REST API routing
express-rate-limit	^7.5.0	7.5.1	Authentication and OTP throttling
express-session	^1.18.1	1.19.0	Cookie-based server sessions
helmet	^8.0.0	8.3.0	HTTP response security headers
multer	^1.4.5-lts.1	1.4.5-lts.2	Multipart attachment upload handling
nodemailer	^6.10.0	6.10.1	Mock/SMTP email transport
nodemon	^3.1.9	3.1.14	Development-time automatic restart

4.1 Programming Language and Runtime

JavaScript is used for the backend and, according to the README, the responsive frontend. The README requires Node.js 20 LTS or newer. Static verification was performed with Node.js v22.16.0, and server.js passed node --check.

4.2 Framework and Database

Express provides routing, middleware composition, static file serving and JSON responses. SQLite is accessed through better-sqlite3. The application enables WAL mode for improved concurrency characteristics and turns on foreign-key enforcement. SQL statements use positional parameters rather than string interpolation for user values.

4.3 Security and Integration Libraries

bcryptjs hashes passwords, Helmet sets security headers, express-rate-limit throttles authentication and OTP endpoints, Multer handles attachments, Nodemailer provides mock or SMTP email delivery, dotenv loads environment values, and express-session maintains authenticated sessions. connect-sqlite3 is listed as a dependency but is not instantiated or imported in server.js; therefore the current code uses the default in-memory session store.

4.4 Development Tools

The manifest supplies npm start for node server.js and npm run dev for nodemon server.js. The README references a local browser at http://localhost:3000. No IDE configuration or version-control metadata was included in the upload, so a specific IDE or Git hosting platform cannot be verified.

5. System Requirements

5.1 Hardware Requirements

Table 5: Hardware requirements

<u>Component</u>	<u>Requirement</u>	<u>Rationale</u>
Processor	64-bit dual-core processor or better	Suitable for Node.js and SQLite development
Memory	4 GB minimum; 8 GB recommended	Allows editor, browser, Node.js and local database concurrently
Storage	At least 1 GB free for source/dependencies; additional space for uploads	node_modules and user attachments grow over time
Network	Required for dependency installation and SMTP mode	Mock email mode can operate offline after installation

5.2 Software Requirements

Table 6: Software requirements

<u>Software</u>	<u>Requirement</u>	<u>Use</u>
Operating system	Windows 10/11, Linux or macOS	Node.js and SQLite are cross-platform
Node.js	20 LTS or newer	Explicit README requirement
npm	Compatible with installed Node.js	Restores dependencies and runs scripts
Web browser	Current Chrome, Edge or Firefox	Required for frontend and portal UI
SQLite tools	Optional	Useful for inspecting cel.sqlite
SMTP account	Optional	Only required when EMAIL_MODE=smtp
Microsoft Word	2019/365 or compatible	For final report editing and submission

6. System Architecture

6.1 High-Level Architecture

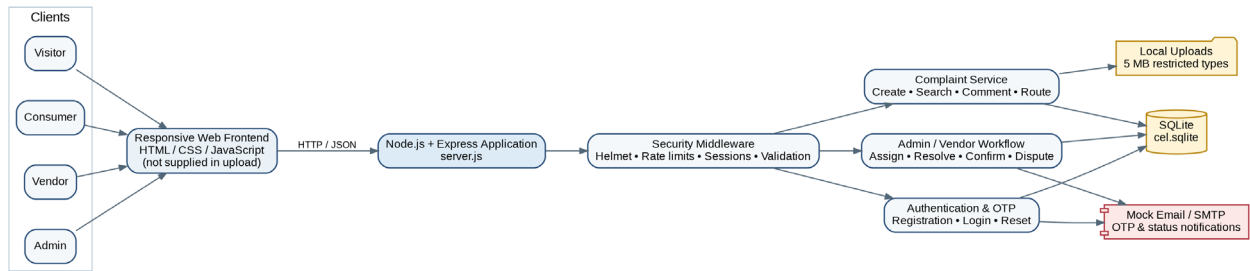


Figure 1: High-level system architecture

The architecture is a monolithic web application: one Express process serves the static frontend and exposes JSON APIs. Domain services share a single SQLite connection. Email is abstracted behind sendMail, while attachments are stored in a local directory. The frontend layer shown in the diagram is documented but absent from the submitted archive.

Table 7: Architecture component responsibilities

Component	Responsibility	Evidence
Browser frontend	Public pages, forms, dashboards and API consumption	README folder structure and Express static middleware
Express application	Routing, validation, authorization and orchestration	server.js routes and helpers
SQLite	Accounts, OTPs, reset codes, complaints and comments	initializeDatabase()
Upload directory	Complaint attachments	Multer diskStorage and /uploads static route
Email service	OTP, password reset and status messages	Nodemailer / mock sendMail
Security middleware	Headers, rate limits, sessions and role guards	Helmet, express-rate-limit, express-session and middleware functions

6.2 Workflow Diagram

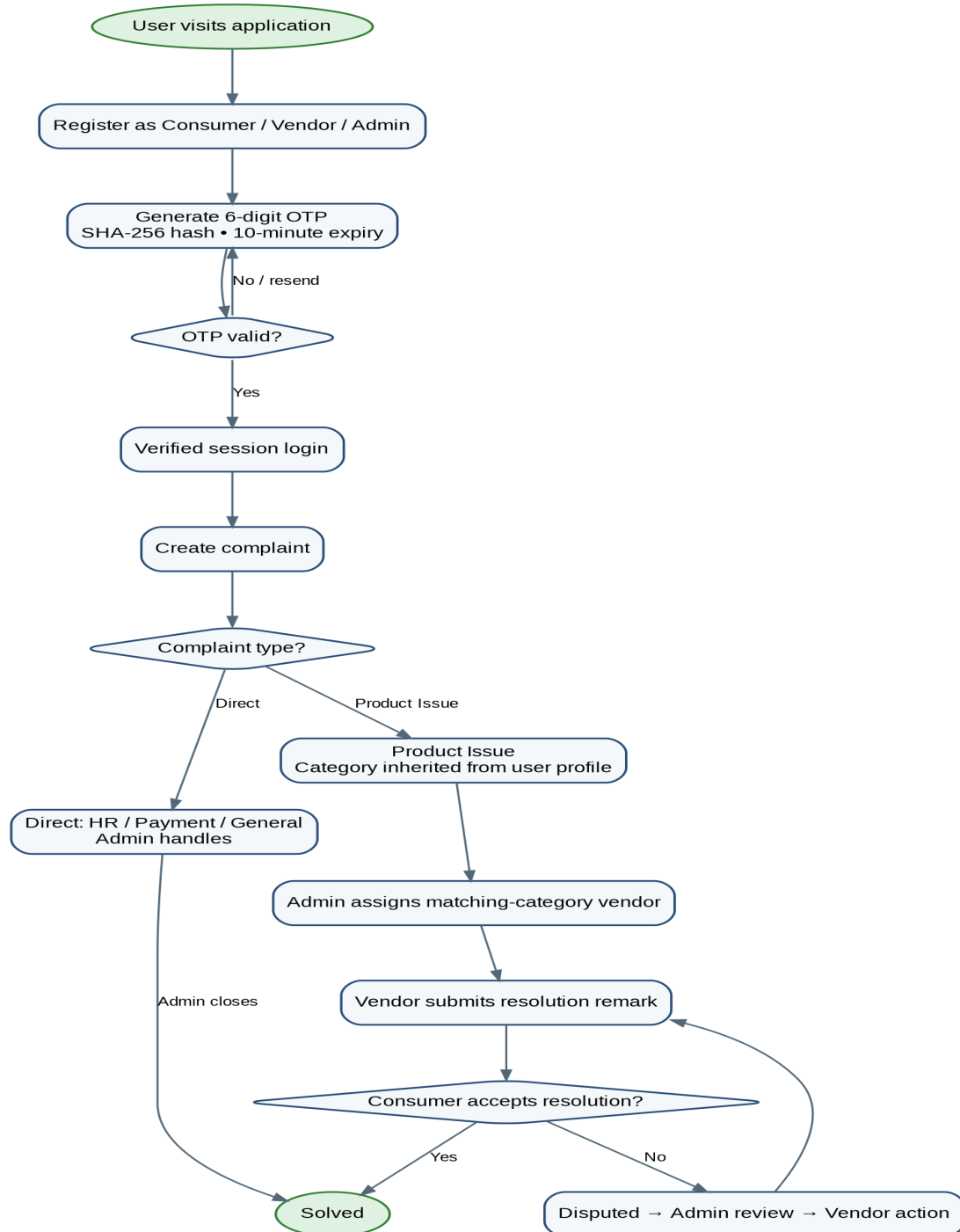


Figure 2: End-to-end complaint workflow

The workflow separates Direct and Product Issue complaints. Direct cases remain under administrator control. Product Issue cases follow a mediated sequence: open, assigned to a category-matched vendor, vendor resolved, and either solved or disputed by the consumer.

6.3 Data Flow Diagram

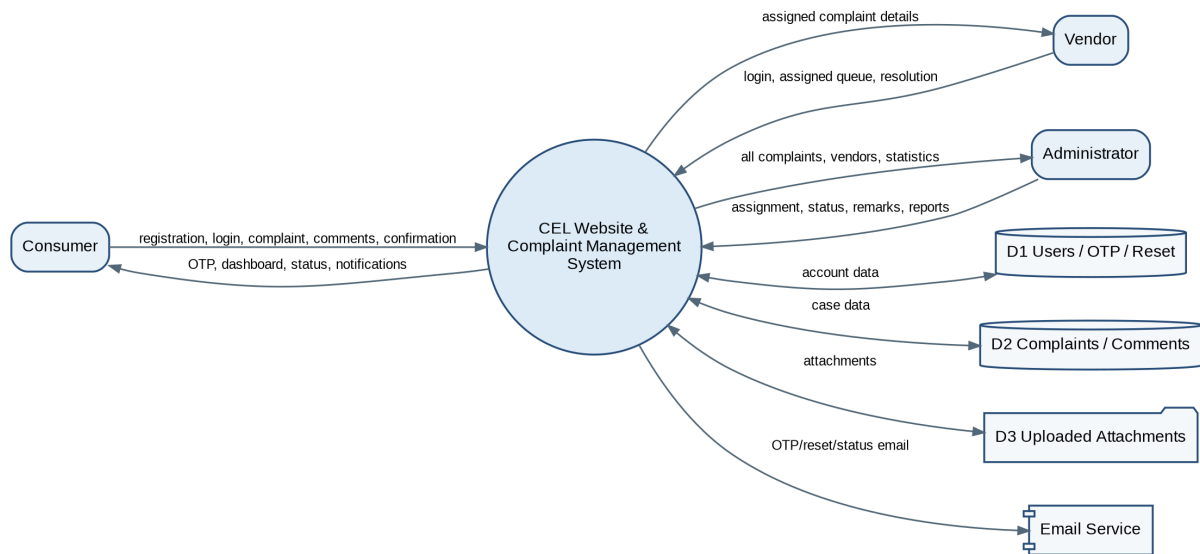


Figure 3: Level-0 data flow diagram

The Level-0 DFD identifies the three authenticated external actors and four data/integration destinations. The system exchanges account and complaint data with SQLite, binary attachments with the file store, and notification messages with the email service.

6.4 Module Diagram

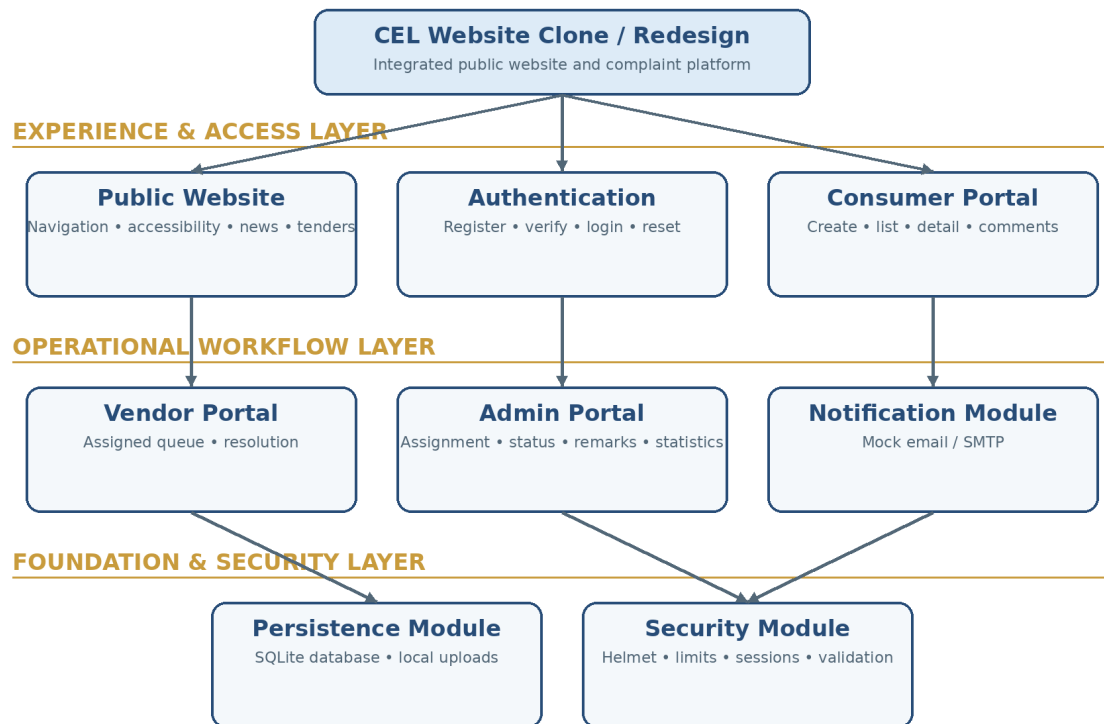


Figure 4: Functional module diagram

6.5 Deployment View

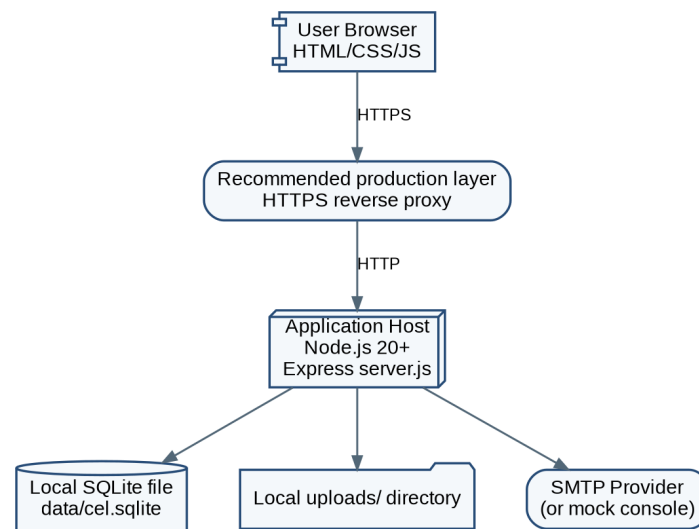


Figure 5: Deployment view and recommended production boundary

The submitted implementation is optimized for local demonstration. A production deployment should place the Node.js host behind HTTPS, replace local uploads with protected object storage, use a managed relational database where scale requires it, and use an approved institutional SMTP provider.

7. Database Design

7.1 Schema Overview and ER Diagram

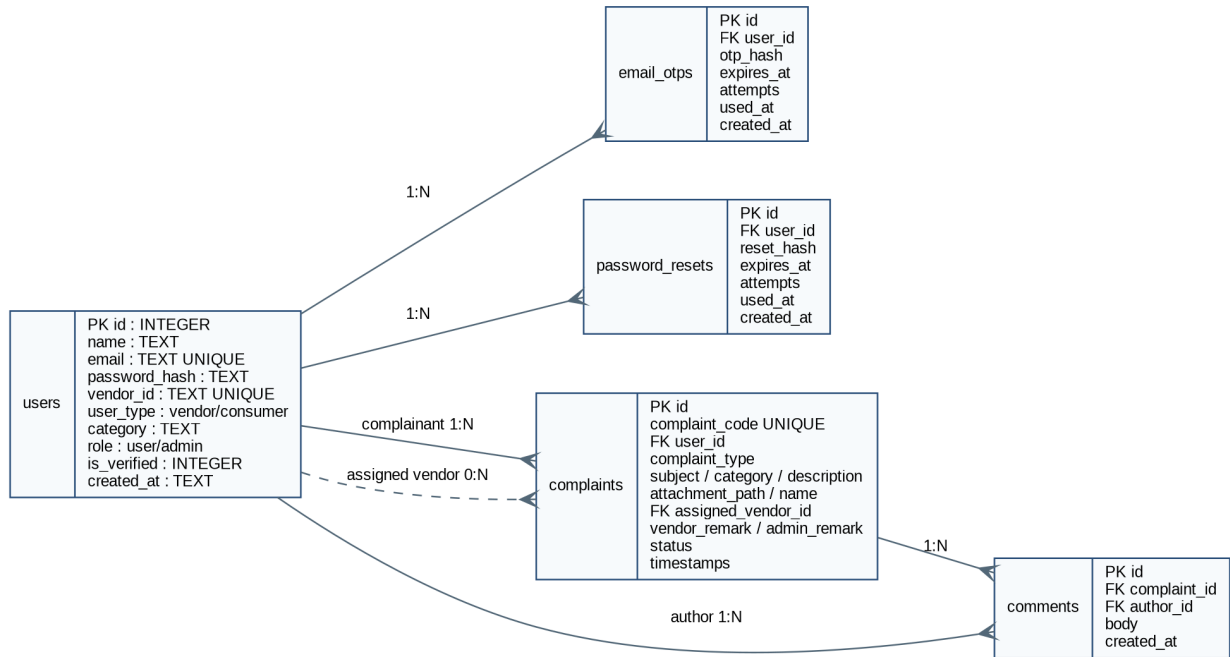


Figure 6: Entity-relationship diagram

The schema contains five application tables. users is the parent entity for authentication identities, complainants, assigned vendors and comment authors. email_otps and password_resets are one-to-many child tables with cascading deletion. complaints references its owner with ON DELETE CASCADE and an optional assigned vendor with ON DELETE SET NULL. comments references both a complaint and an author with cascading deletion.

Table 8: Database relationships

Relationship	Cardinality	Foreign key	Delete behavior
users → email_otps	1:N	email_otps.user_id	ON DELETE CASCADE
users → password_resets	1:N	password_resets.user_id	ON DELETE CASCADE
users → complaints (owner)	1:N	complaints.user_id	ON DELETE CASCADE
users → complaints (vendor)	1:N optional	complaints.assigned_vendor_id	ON DELETE SET NULL
complaints → comments	1:N	comments.complaint_id	ON DELETE CASCADE
users → comments	1:N	comments.author_id	ON DELETE CASCADE

7.2 Users Table

Table 9: users table data dictionary

Field	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Internal identity
name	TEXT	NOT NULL	Full display name
email	TEXT	NOT NULL, UNIQUE, NOCASE	Login and notification address
password_hash	TEXT	NOT NULL	bcrypt hash; plaintext is not stored
vendor_id	TEXT	UNIQUE, NOCASE, nullable	Seller/vendor registration code
user_type	TEXT	CHECK vendor/consumer	Operational user type
category	TEXT	Nullable	Manufacturing category
role	TEXT	CHECK user/admin	Authorization role
is_verified	INTEGER	DEFAULT 0	Email verification flag
created_at	TEXT	DEFAULT CURRENT_TIMESTAMP	Creation time

7.3 email_otp Table

Table 10: email_otp table data dictionary

Field	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	OTP record identity
user_id	INTEGER	FK users.id, NOT NULL	Account being verified
otp_hash	TEXT	NOT NULL	SHA-256 digest of six-digit OTP
expires_at	TEXT	NOT NULL	Ten-minute validity boundary
attempts	INTEGER	DEFAULT 0	Incorrect attempts; maximum five
used_at	TEXT	Nullable	Marks successful consumption
created_at	TEXT	DEFAULT timestamp	Record creation time

7.4 password_resets Table

Table 11: password_resets table data dictionary

Field	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Reset record identity
user_id	INTEGER	FK users.id, NOT NULL	Account requesting reset
reset_hash	TEXT	NOT NULL	SHA-256 digest of reset code
expires_at	TEXT	NOT NULL	Ten-minute validity boundary
attempts	INTEGER	DEFAULT 0	Incorrect attempts; maximum five

<u>Field</u>	<u>Type</u>	<u>Constraints</u>	<u>Description</u>
used_at	TEXT	Nullable	Marks successful password change
created_at	TEXT	DEFAULT timestamp	Record creation time

7.5 Complaints Table

Table 12: complaints table data dictionary

<u>Field</u>	<u>Type</u>	<u>Constraints</u>	<u>Description</u>
id	INTEGER	PK, AUTOINCREMENT	Internal complaint identity
complaint_code	TEXT	UNIQUE	Human-readable CEL-YYYY-NNNNNN ID
user_id	INTEGER	FK users.id, NOT NULL	Complaint owner
complaint_type	TEXT	CHECK Product Issue/Direct	Routing model
subject	TEXT	NOT NULL	Short issue title
category	TEXT	NOT NULL	Manufacturing or Direct category
description	TEXT	NOT NULL	Detailed issue description
attachment_path	TEXT	Nullable	Randomized stored filename
attachment_name	TEXT	Nullable	Original client filename
assigned_vendor_id	INTEGER	FK users.id, nullable	Vendor selected by administrator
vendor_remark	TEXT	Nullable	Vendor resolution explanation
vendor_resolved_at	TEXT	Nullable	Vendor resolution timestamp
consumer_confirmed_at	TEXT	Nullable	Final confirmation timestamp
status	TEXT	CHECK five statuses	Lifecycle state
admin_remark	TEXT	Nullable	Official administrator remark
created_at	TEXT	Default timestamp	Creation time
updated_at	TEXT	Default timestamp	Last activity time

7.6 Comments Table

Table 13: comments table data dictionary

Field	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Comment identity
complaint_id	INTEGER	FK complaints.id	Parent complaint
author_id	INTEGER	FK users.id	Consumer, vendor or admin author
body	TEXT	NOT NULL	Comment text, limited by API to 1,500 characters
created_at	TEXT	Default timestamp	Comment time

7.7 Constraints, Indexes and Migrations

- users.email and users.vendor_id are case-insensitive unique fields.
- CHECK constraints restrict user_type, role, complaint_type and complaint status values.
- Foreign-key enforcement is explicitly enabled with PRAGMA foreign_keys = ON.
- WAL journal mode is enabled for database durability and concurrency characteristics.
- idx_complaints_user accelerates owner-based complaint queries.
- idx_comments_complaint accelerates comment-thread retrieval.
- The code adds user_type and category columns to older user databases when missing.
- An older complaints table is rebuilt into the vendor-routing schema, with legacy rows preserved as Direct/General and prior category information embedded into the description.

7.8 Complaint State Model

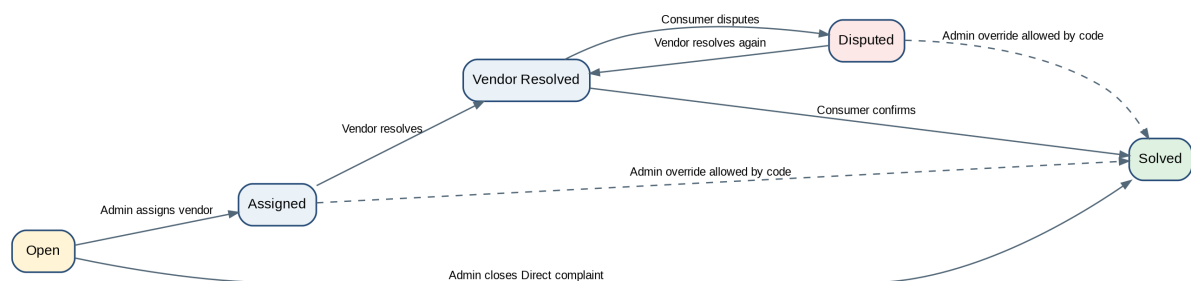


Figure 7: Complaint state-transition model

Direct complaints are constrained at the API layer to Open or Solved. Product Issue complaints may occupy any of the five database states. The administrator endpoint can set any listed status for Product Issue cases, while vendor and consumer endpoints enforce the normal operational sequence.

8. Module Description

Table 14: Module input-output summary

Module	Purpose / working	Inputs	Outputs
Application bootstrap	Create runtime directories, open SQLite, initialize/migrate schema and seed admin	Environment variables, filesystem	Ready server and database
Public web module	Serve public assets and expose verified vendor list	Browser request, database vendors	HTML assets / vendor JSON
Registration & OTP	Validate identity, create account, issue/verify OTP	Name, email, password, type, category	Verified account
Login & session	Authenticate password and establish session	Email and password	Session user object
Password reset	Issue generic reset response and validate code	Email, code, new password	Updated bcrypt hash
Complaint creation	Validate type/category/text and save optional file	Authenticated form and attachment	Complaint ID
Complaint query	List, filter, paginate, summarize and read details	Scope, page, status, category, search	Complaint JSON
Comments	Add conversation entries with access checks	Complaint ID and text	Persistent comment
Admin mediation	Assign matching vendor, update status/remark, view stats	Admin session and update payload	Updated complaint / statistics
Vendor resolution	Resolve only assigned cases	Vendor session and remark	Vendor Resolved status
Consumer closure	Confirm or dispute vendor resolution	Owner session	Solved or Disputed status
Notification	Send OTP, reset and status emails	Recipient and message data	Mock console or SMTP delivery
Security & upload	Headers, limits, role guards and file restrictions	HTTP requests and files	Accepted/rejected request

8.1 Authentication Module

Registration validates full name, email syntax, password strength, role intent, user type, manufacturing category and optional vendor ID. The password is hashed before insertion. A random six-digit OTP is generated with `crypto.randomInt`, hashed with SHA-256 and stored with a ten-minute expiry. Verification compares hashes, increments incorrect attempts and uses a transaction to verify the user and consume the OTP atomically.

8.2 Complaint and Routing Module

The complaint module distinguishes Product Issue and Direct cases. Product Issue category is taken from `req.session.user.category` rather than `req.body`, which prevents client manipulation. Direct category must be one of HR, Payment or General. The inserted row receives a generated code based on the current year and database row ID.

8.3 Administration, Vendor and Consumer Modules

The administrator is the mediator. The assignment route validates that the selected account is a vendor in the complaint category. Vendors can act only on complaints assigned to their user ID and only while the case is Assigned or Disputed. Consumers can confirm or dispute only their own Vendor Resolved cases. These checks implement separation of duties and prevent unauthorized workflow transitions.

8.4 Security Module

Security controls include HTTP-only, SameSite=Lax session cookies, secure cookies in production, an eight-hour maximum age, Helmet headers, separate authentication and OTP rate limits, server-side role middleware, length limits, basic angle-bracket stripping, parameterized queries, password hashing, OTP hashing, file size limits and an extension whitelist. Gaps are discussed in Sections 17 and 18.

9. Source Code Explanation

9.1 Important Functions

Table 15: Important functions and responsibilities

Function	Responsibility	Lines
initializeDatabase()	Creates five tables and two indexes, performs migrations and seeds the administrator	39–184
cleanText()	Removes angle brackets, trims and truncates input	226–228
strongPassword()	Requires 8+ characters with uppercase, lowercase and digit	231–233
requireUser()	Rejects requests without a session	234–237
requireAdmin()	Requires role=admin	238–241
requireVendor()	Requires userType=vendor	242–245
publicUser()	Returns session-safe user fields	246–257
generateOtp()/hashOtp()	Creates six-digit code and SHA-256 digest	258–259
sendMail()	Selects mock console mode or SMTP transport	261–274
sendOtpEmail()/sendResetEmail()/sendStatusEmail()	Compose notification messages	276–301

9.2 Line-by-Line Explanation of Key Logic

Table 16: Source-code traceability by line range

Lines	Explanation
22–24	Opens data/cel.sqlite, enables WAL and foreign keys.
41–109	Defines users, OTP, reset, complaints and comments tables plus indexes.
170–181	Seeds one verified administrator using configured values when absent.
190–200	Configures server-side session middleware and cookie attributes.
204–205	Defines rate limits: 30 auth requests/15 min and 8 OTP requests/10 min.
207–223	Randomizes stored upload names and restricts extensions and file size.
317–359	Performs registration validation, bcrypt hash, user insert and OTP delivery.
366–383	Verifies OTP expiry, attempts and hash, then commits verification transaction.
417–457	Implements reset code generation and password replacement.
468–498	Builds role-aware complaint filters with pagination and parameter binding.
515–540	Creates complaint and assigns CEL-YYYY-NNNNNN identifier.
571–599	Allows admin assignment/status/remark updates and sends status email.

<u>Lines</u>	<u>Explanation</u>
602–632	Implements vendor resolution followed by consumer confirm/dispute.
643–653	Computes administrator statistics with aggregate SQL queries.

9.3 API Design

Routes follow a REST-like JSON design. Authentication endpoints are grouped under `/api/auth`, complaint operations under `/api/complaints`, public vendor discovery under `/api/public`, administrative operations under `/api/admin` and vendor resolution under `/api/vendor`. The API returns explicit HTTP status codes such as 201 for creation, 400 for validation failures, 401 for unauthenticated requests, 403 for authorization failures, 404 for missing resources, 409 for uniqueness conflicts and 429 for excessive attempts.

9.4 Code Quality Assessment

- Strengths: cohesive single-file prototype, parameterized queries, clear route grouping, comments explaining domain rules, database constraints, migrations and explicit access checks.
- Maintainability concern: `server.js` contains database setup, mail, validation and all routes in 665 lines; production code should be separated into controllers, services, repositories, middleware and configuration modules.
- Documentation mismatch: README claims a SQLite-backed session store, but `connect-sqlite3` is not used.
- Security concern: uploaded files are served by a public static route; authorization is not checked at download time.
- Security concern: Content Security Policy is disabled in Helmet, and CSRF protection is absent.
- Testing concern: no automated test framework or test files were included.

10. Project Folder Structure

10.1 Uploaded Artifact Inventory

```

Uploaded artifact inventory – verified files and omissions

/mnt/data (uploaded project artifacts)
├── .env
├── .env(1).example
├── package(1).json
├── package-lock.json
├── README(1).md
└── server(1).js

Not present in the upload:
├── public/index.html
├── public/css/styles.css
├── public/js/app.js
├── public/assets/
├── data/
└── uploads/

```

Figure 8: Uploaded artifact inventory and verified omissions

Only six project artifacts were received. The actual filenames contain suffixes introduced during upload, but their contents correspond to the environment file, example environment file, package manifest, lockfile, README and server source. No database file, upload samples, frontend assets, screenshots or version-control metadata were supplied.

10.2 Documented Intended Structure

```

Documented intended project hierarchy

cel-clone/
├── data/                # SQLite DB and session DB (runtime)
├── public/
│   ├── assets/
│   ├── css/styles.css
│   ├── js/app.js
│   └── index.html
├── uploads/            # complaint attachments
├── .env.example
├── package.json
├── server.js
└── README.md

Source: README.md, lines 70–84.

```

Figure 9: Documented intended project hierarchy

Table 17: Uploaded versus documented folder structure

Path	Purpose	Evidence status
data/	Documented runtime SQLite database and session database directory	Not supplied; schema database generated during analysis
public/assets/	Images and other public assets	Missing
public/css/styles.css	Responsive frontend styling	Missing
public/js/app.js	Frontend behavior and API calls	Missing
public/index.html	Single-page user interface entry point	Missing
uploads/	Complaint attachments	Not supplied; created at runtime by server

<u>Path</u>	<u>Purpose</u>	<u>Evidence status</u>
.env / .env.example	Runtime and example configuration	Supplied
package.json / lockfile	Dependency and script definition	Supplied
server.js	Backend and schema implementation	Supplied
README.md	Project documentation	Supplied

11. User Interface

Evidence limitation — The README describes the user interface, but the public/ folder and application screenshots were not included. The backend's wildcard route expects public/index.html; therefore authentic UI pages cannot be rendered from the received upload. No substitute or invented screenshots are included.

Table 18: UI screenshot evidence status

Screen	Verified basis	Required screenshot
Public home page	README documents header, navigation, hero, business cards, CMD message, news, tenders, products and footer	Capture full desktop and responsive mobile views
Accessibility controls	Font size, grayscale, high contrast, underlined links, readable font and reset are documented	Capture default and each enabled mode
Registration page	Inputs inferred from API: name, email, password, user type, category, vendor ID/admin code	Capture consumer and vendor variants
OTP verification	API returns email and mock OTP in development	Capture verification and resend states
Login / forgot password	API routes implemented	Capture login, forgot-code and reset screens
Consumer dashboard	Complaint summary, personal list, filters and detail thread expected	Capture empty, populated and paginated states
Complaint form	Product Issue versus Direct category behavior and attachment field	Capture both complaint types
Vendor dashboard	Assigned queue and resolution remark expected	Capture Assigned and Disputed cases
Admin dashboard	Statistics, complaint table, vendor assignment, remarks and status controls expected	Capture overview and detail workflow
Approved vendor list	Public endpoint exists and exposes name, category and vendor ID	Capture grouped vendor list

11.1 Screenshot Capture Procedure

1. Restore the documented public/ directory from the original project source.
2. Run npm install, copy .env.example to .env and replace development secrets.
3. Start the application with npm start and open http://localhost:3000.
4. Use mock email mode for OTP demonstrations; redact OTPs and credentials in final screenshots.
5. Create one consumer, one vendor in the same manufacturing category and one separate vendor in a different category.
6. Exercise the full complaint lifecycle and capture each state at 100% browser zoom.
7. Use consistent 16:9 desktop screenshots and include one responsive mobile viewport set.

12. Coding Screenshots

This mandatory section contains screenshots generated directly from the uploaded source files with original line numbers. No code was rewritten for the figures.

Table 19: Coding screenshot index

Figure	File name	Coverage
Figure 10	code01_bootstrap.png	Application bootstrap and domain constants
Figure 11	code02_schema.png	SQLite tables, constraints and indexes
Figure 12	code03_security_uploads.png	Admin seeding, session configuration, security middleware and uploads
Figure 13	code04_helpers_email.png	Validation, role guards and email services
Figure 14	code05_auth.png	Registration, OTP verification and login
Figure 15	code06_reset_queries.png	Password reset and complaint query model
Figure 16	code07_complaint_create.png	Complaint listing, summary and creation
Figure 17	code08_admin_flow.png	Complaint detail, comments and admin mediation
Figure 18	code09_vendor_stats.png	Vendor resolution, consumer confirmation and admin statistics
Figure 19	code10_package.png	Declared package dependencies and scripts
Figure 20	code11_readme_features.png	Documented functional and security features
Figure 21	code12_readme_structure_run.png	Documented hierarchy and execution instructions
Figure 22	code13_env_example.png	Environment-variable interface using development example values

server.js — application bootstrap and domain constants

```

1 require('dotenv').config();
2 const path = require('path');
3 const fs = require('fs');
4 const crypto = require('crypto');
5 const express = require('express');
6 const session = require('express-session');
7 const Database = require('better-sqlite3');
8 const bcrypt = require('bcryptjs');
9 const multer = require('multer');
10 const nodemailer = require('nodemailer');
11 const helmet = require('helmet');
12 const rateLimit = require('express-rate-limit');
13
14 const app = express();
15 const PORT = Number(process.env.PORT || 3000);
16 const ROOT = __dirname;
17 const DATA_DIR = path.join(ROOT, 'data');
18 const UPLOAD_DIR = path.join(ROOT, 'uploads');
19 fs.mkdirSync(DATA_DIR, { recursive: true });
20 fs.mkdirSync(UPLOAD_DIR, { recursive: true });
21
22 const db = new Database(path.join(DATA_DIR, 'cel.sqlite'));
23 db.pragma('journal_mode = WAL');
24 db.pragma('foreign_keys = ON');
25
26 // The four manufacturing verticals every vendor and consumer must register under.
27 // Product Issue complaints are always routed within a single one of these categories.
28 const MANUFACTURE_CATEGORIES = [
29   'Defence & Strategic Systems',
30   'Industrial Electronics & IT',
31   'Railway Technologies',
32   'Renewable Energy'
33 ];
34 // Complaints that never involve a vendor and stay with admin only.
35 const DIRECT_CATEGORIES = ['HR', 'Payment', 'General'];
36 const COMPLAINT_TYPES = ['Product Issue', 'Direct'];
37 const COMPLAINT_STATUSES = ['Open', 'Assigned', 'Vendor Resolved', 'Disputed', 'Solved'];

```

Figure 10: Application bootstrap and domain constants

Imports built-in and third-party modules, establishes runtime paths and defines the four manufacturing categories, Direct categories and five complaint statuses.

server.js — SQLite tables, keys and indexes

```

39 function initializeDatabase() {
40   db.exec(`
41     CREATE TABLE IF NOT EXISTS users (
42       id INTEGER PRIMARY KEY AUTOINCREMENT,
43       name TEXT NOT NULL,
44       email TEXT NOT NULL UNIQUE COLLATE NOCASE,
45       password_hash TEXT NOT NULL,
46       vendor_id TEXT UNIQUE COLLATE NOCASE,
47       user_type TEXT NOT NULL DEFAULT 'consumer' CHECK(user_type IN ('vendor','consumer')),
48       category TEXT,
49       role TEXT NOT NULL DEFAULT 'user' CHECK(role IN ('user','admin')),
50       is_verified INTEGER NOT NULL DEFAULT 0,
51       created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
52     );
53
54     CREATE TABLE IF NOT EXISTS email_otps (
55       id INTEGER PRIMARY KEY AUTOINCREMENT,
56       user_id INTEGER NOT NULL,
57       otp_hash TEXT NOT NULL,
58       expires_at TEXT NOT NULL,
59       attempts INTEGER NOT NULL DEFAULT 0,
60       used_at TEXT,
61       created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
62       FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE
63     );
64
65     CREATE TABLE IF NOT EXISTS password_resets (
66       id INTEGER PRIMARY KEY AUTOINCREMENT,
67       user_id INTEGER NOT NULL,
68       reset_hash TEXT NOT NULL,
69       expires_at TEXT NOT NULL,
70       attempts INTEGER NOT NULL DEFAULT 0,
71       used_at TEXT,
72       created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
73       FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE
74     );
75
76     CREATE TABLE IF NOT EXISTS complaints (
77       id INTEGER PRIMARY KEY AUTOINCREMENT,
78       complaint_code TEXT UNIQUE,
79       user_id INTEGER NOT NULL,
80       complaint_type TEXT NOT NULL DEFAULT 'Direct' CHECK(complaint_type IN ('Product Issue','Direct')),
81       subject TEXT NOT NULL,
82       category TEXT NOT NULL,
83       description TEXT NOT NULL,
84       attachment_path TEXT,
85       attachment_name TEXT,
86       assigned_vendor_id INTEGER,
87       vendor_remark TEXT,
88       vendor_resolved_at TEXT,
89       consumer_confirmed_at TEXT,
90       status TEXT NOT NULL DEFAULT 'Open' CHECK(status IN ('Open','Assigned','Vendor Resolved','Disputed','Solved')),
91       admin_remark TEXT,
92       created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
93       updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
94       FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE,
95       FOREIGN KEY(assigned_vendor_id) REFERENCES users(id) ON DELETE SET NULL
96     );
97
98     CREATE TABLE IF NOT EXISTS comments (
99       id INTEGER PRIMARY KEY AUTOINCREMENT,
100      complaint_id INTEGER NOT NULL,
101      author_id INTEGER NOT NULL,
102      body TEXT NOT NULL,
103      created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
104      FOREIGN KEY(complaint_id) REFERENCES complaints(id) ON DELETE CASCADE,
105      FOREIGN KEY(author_id) REFERENCES users(id) ON DELETE CASCADE
106     );
107
108     CREATE INDEX IF NOT EXISTS idx_complaints_user ON complaints(user_id);
109     CREATE INDEX IF NOT EXISTS idx_comments_complaint ON comments(complaint_id);

```

Figure 11: SQLite tables, constraints and indexes

Shows the complete initial SQL schema with primary keys, unique constraints, CHECK constraints, foreign keys and indexes.

server.js — admin seeding, sessions, security and uploads

```

170 const adminEmail = (process.env.ADMIN_EMAIL || 'admin@cel.local').trim().toLowerCase();
171 const existing = db.prepare('SELECT id FROM users WHERE email = ?').get(adminEmail);
172 if (!existing) {
173   const hash = bcrypt.hashSync(process.env.ADMIN_PASSWORD || 'Admin@12345', 12);
174   db.prepare('INSERT INTO users(name,email,password_hash,role,is_verified)
175     VALUES(?,?,?,?,1)').run(
176     process.env.ADMIN_NAME || 'CEL Administrator',
177     adminEmail,
178     hash,
179     'admin'
180   );
181   console.log(`Seeded admin account: ${adminEmail}`);
182 }
183 }
184 initializeDatabase();
185
186 app.set('trust proxy', 1);
187 app.use(helmet({ contentSecurityPolicy: false, crossOriginResourcePolicy: false }));
188 app.use(express.json({ limit: '1mb' }));
189 app.use(express.urlencoded({ extended: false }));
190 app.use(session({
191   secret: process.env.SESSION_SECRET || 'development-only-change-this-secret',
192   resave: false,
193   saveUninitialized: false,
194   cookie: {
195     httpOnly: true,
196     sameSite: 'lax',
197     secure: process.env.NODE_ENV === 'production',
198     maxAge: 1000 * 60 * 60 * 8
199   }
200 }));
201 app.use('/uploads', express.static(UPLOAD_DIR, { dotfiles: 'deny', index: false }));
202 app.use(express.static(path.join(ROOT, 'public')));
203
204 const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, limit: 30, standardHeaders: true, legacyHeaders: false });
205 const otpLimiter = rateLimit({ windowMs: 10 * 60 * 1000, limit: 8, standardHeaders: true, legacyHeaders: false });
206
207 const allowedExtensions = new Set(['.pdf', '.png', '.jpg', '.jpeg', '.doc', '.docx']);
208 const storage = multer.diskStorage({
209   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
210   filename: (_req, file, cb) => {
211     const ext = path.extname(file.originalname).toLowerCase();
212     cb(null, `${Date.now()}-${crypto.randomBytes(8).toString('hex')}${ext}`);
213   }
214 });
215 const upload = multer({
216   storage,
217   limits: { fileSize: 5 * 1024 * 1024 },
218   fileFilter: (_req, file, cb) => {
219     const ext = path.extname(file.originalname).toLowerCase();
220     if (!allowedExtensions.has(ext)) return cb(new Error('Unsupported attachment type.'));
221     cb(null, true);
222   }
223 });

```

Figure 12: Admin seeding, session configuration, security middleware and uploads

Shows administrator seeding, Express parsing, session cookie settings, rate limits and Multer attachment restrictions.

server.js — validation, authorization helpers and email services

```

225 function normalizeEmail(value) { return String(value || '').trim().toLowerCase(); }
226 function cleanText(value, max = 5000) {
227   return String(value || '').replace(/[<>]/g, '').trim().slice(0, max);
228 }
229 function validEmail(email) { return /^[^@]+\.[^@]+\.[^@]+\.[^@]+$/i.test(email); }
230 function validVendorId(value) { return /^[A-Za-z0-9-]{3,30}$/i.test(String(value || '').trim()); }
231 function strongPassword(password) {
232   return typeof password === 'string' && password.length >= 8 && /[A-Z]/.test(password) && /[a-z]/.test(password) && /\d/.test(password);
233 }
234 function requireUser(req, res, next) {
235   if (!req.session.user) return res.status(401).json({ error: 'Please log in first.' });
236   next();
237 }
238 function requireAdmin(req, res, next) {
239   if (!req.session.user || req.session.user.role !== 'admin') return res.status(403).json({ error: 'Admin access required.' });
240   next();
241 }
242 function requireVendor(req, res, next) {
243   if (!req.session.user || req.session.user.userType !== 'vendor') return res.status(403).json({ error: 'Vendor access required.' });
244   next();
245 }
246 function publicUser(user) {
247   return user ? {
248     id: user.id,
249     name: user.name,
250     email: user.email,
251     vendorId: user.vendor_id,
252     userType: user.user_type,
253     category: user.category,
254     role: user.role,
255     isVerified: Boolean(user.is_verified)
256   } : null;
257 }
258 function generateOtp() { return String(crypto.randomInt(100000, 1000000)); }
259 function hashOtp(otp) { return crypto.createHash('sha256').update(otp).digest('hex'); }
260
261 async function sendMail(to, subject, text, html) {
262   if ((process.env.EMAIL_MODE || 'mock').toLowerCase() !== 'smtp') {
263     console.log([MOCK EMAIL] To: ${to} | Subject: ${subject} | ${text});
264     return { mock: true };
265   }
266   const transporter = nodemailer.createTransport({
267     host: process.env.SMTP_HOST,
268     port: Number(process.env.SMTP_PORT || 587),
269     secure: String(process.env.SMTP_SECURE).toLowerCase() === 'true',
270     auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
271   });
272   await transporter.sendMail({ from: process.env.EMAIL_FROM || process.env.SMTP_USER, to, subject, text, html });
273   return { mock: false };
274 }
275
276 async function sendOtpEmail(email, otp) {
277   return sendMail(
278     email,
279     'CEL account verification OTP',
280     `Your CEL verification OTP is ${otp}. It expires in 10 minutes.`,
281     `<p>Your CEL verification OTP is <strong>${otp}</strong>.</p><p>It expires in 10 minutes.</p>`
282   );
283 }
284
285 async function sendResetEmail(email, code) {
286   return sendMail(
287     email,
288     'CEL password reset code',
289     `Your CEL password reset code is ${code}. It expires in 10 minutes. If you didn't request this, you can ignore this email.`,
290     `<p>Your CEL password reset code is <strong>${code}</strong>.</p><p>It expires in 10 minutes. If you didn't request this, you can ignore this email.</p>`
291   );
292 }
293
294 async function sendStatusEmail(email, complaintCode, status, remark) {
295   return sendMail(
296     email,
297     `Your CEL complaint ${complaintCode} was updated`,
298     `Your complaint ${complaintCode} is now marked: ${status}. ${remark ? `Remark: ${remark}` : ''}`,
299     `<p>Your complaint <strong>${complaintCode}</strong> is now marked: <strong>${status}</strong>.</p><p>${remark ? `<p>Remark: ${remark}</p>` : ''}`
300   );
301 }

```

Figure 13: Validation, role guards and email services

Shows input normalization, authorization middleware, safe session-user projection, OTP hashing and mock/SMTP email abstraction.

server.js — registration, OTP verification and login

```

317 app.post('/api/auth/register', authLimiter, async (req, res) => {
318   try {
319     const name = cleanText(req.body.name, 80);
320     const email = normalizeEmail(req.body.email);
321     const password = String(req.body.password || '');
322     const adminCode = String(req.body.adminCode || '').trim();
323     if (adminCode && (!process.env.ADMIN_SIGNUP_CODE || adminCode !== process.env.ADMIN_SIGNUP_CODE)) {
324       return res.status(403).json({ error: 'Incorrect admin access code.' });
325     }
326     const wantsAdmin = Boolean(adminCode);
327
328     if (name.length < 2) return res.status(400).json({ error: 'Please enter your full name.' });
329     if (!validEmail(email)) return res.status(400).json({ error: 'Enter a valid email address.' });
330     if (!strongPassword(password)) return res.status(400).json({ error: 'Password must be at least 8 characters and include uppercase, lowercase and a number.' });
331     if (db.prepare('SELECT id FROM users WHERE email=?').get(email)) return res.status(409).json({ error: 'An account with this email already exists.' });
332
333     let vendorId = null;
334     let userType = 'consumer';
335     let category = null;
336     if (!wantsAdmin) {
337       userType = String(req.body.userType || '').trim().toLowerCase();
338       if (!['vendor', 'consumer'].includes(userType)) {
339         return res.status(400).json({ error: 'Select whether you are registering as a Vendor/Seller or a Consumer.' });
340       }
341       category = cleanText(req.body.category, 60);
342       if (!MANUFACTURE_CATEGORIES.includes(category)) {
343         return res.status(400).json({ error: 'Select the CEL manufacture category you belong to.' });
344       }
345       if (userType === 'vendor') {
346         vendorId = cleanText(req.body.vendorId, 30).toUpperCase();
347         if (!validVendorId(vendorId)) return res.status(400).json({ error: 'Enter a valid Seller/Vendor ID (3-30 letters, numbers or hyphens).' });
348         if (db.prepare('SELECT id FROM users WHERE vendor_id=?').get(vendorId)) return res.status(409).json({ error: 'This Seller/Vendor ID is already registered.' });
349       }
350     }
351
352     const role = wantsAdmin ? 'admin' : 'user';
353     const result = db.prepare('INSERT INTO users (name,email,password_hash,vendor_id,user_type,category,role) VALUES(?, ?, ?, ?, ?, ?, ?)')
354     .run(name, email, bcrypt.hashSync(password, 12), vendorId, userType, category, role);
355     const otp = generateOtp();
356     db.prepare('INSERT INTO email_otps (user_id,otp_hash,expires_at) VALUES(?, ?, datetime(\'now\',\'+10 minutes\'))')
357     .run(result.lastInsertRowid, hashOtp(otp));
358     const delivery = await sendOtpEmail(email, otp);
359     res.status(201).json({ message: 'Registration successful. Verify your email with the OTP.', email, mockOtp: delivery.mock ? otp : undefined });
360   } catch (error) {
361     console.error(error);
362     res.status(500).json({ error: 'Could not register the account.' });
363   }
364 });
365
366 app.post('/api/auth/verify-otp', otpLimiter, (req, res) => {
367   const email = normalizeEmail(req.body.email);
368   const otp = cleanText(req.body.otp, 6);
369   const user = db.prepare('SELECT * FROM users WHERE email=?').get(email);
370   if (!user) return res.status(404).json({ error: 'Account not found.' });
371   const record = db.prepare('SELECT * FROM email_otps WHERE user_id=? AND used_at IS NULL ORDER BY id DESC LIMIT 1').get(user.id);
372   if (!record) return res.status(400).json({ error: 'No active OTP found.' });
373   if (new Date(record.expires_at.replace('T', 'Z')) < new Date()) return res.status(400).json({ error: 'OTP has expired.' });
374   if (record.attempts >= 5) return res.status(429).json({ error: 'Too many incorrect attempts. Request a new OTP.' });
375   if (hashOtp(otp) !== record.otp_hash) {
376     db.prepare('UPDATE email_otps SET attempts=attempts+1 WHERE id=?').run(record.id);
377     return res.status(400).json({ error: 'Incorrect OTP.' });
378   }
379   db.transaction(() => {
380     db.prepare('UPDATE users SET is_verified=1 WHERE id=?').run(user.id);
381     db.prepare('UPDATE email_otps SET used_at=CURRENT_TIMESTAMP WHERE id=?').run(record.id);
382   })();
383   res.json({ message: 'Email verified. You may now log in.' });
384 });
385
386 app.post('/api/auth/resend-otp', otpLimiter, async (req, res) => {
387   try {
388     const email = normalizeEmail(req.body.email);
389     const user = db.prepare('SELECT * FROM users WHERE email=?').get(email);
390     if (!user) return res.status(404).json({ error: 'Account not found.' });
391     if (user.is_verified) return res.status(400).json({ error: 'This account is already verified.' });
392     const otp = generateOtp();
393     db.prepare('INSERT INTO email_otps (user_id,otp_hash,expires_at) VALUES(?, ?, datetime(\'now\',\'+10 minutes\'))').run(user.id, hashOtp(otp));
394     const delivery = await sendOtpEmail(email, otp);
395     res.json({ message: 'A new OTP has been sent.', mockOtp: delivery.mock ? otp : undefined });
396   } catch (error) {
397     console.error(error);
398     res.status(500).json({ error: 'Could not resend OTP.' });
399   }
400 });
401
402 app.post('/api/auth/login', authLimiter, (req, res) => {
403   const email = normalizeEmail(req.body.email);
404   const password = String(req.body.password || '');
405   const user = db.prepare('SELECT * FROM users WHERE email=?').get(email);
406   if (!user || !bcrypt.compareSync(password, user.password_hash)) return res.status(401).json({ error: 'Incorrect email or password.' });
407   if (!user.is_verified) return res.status(403).json({ error: 'Verify your email before logging in.', needsVerification: true, email });
408   req.session.regenerate(err => {
409     if (err) return res.status(500).json({ error: 'Could not start session.' });
410     req.session.user = publicUser(user);
411     res.json({ message: 'Logged in successfully.', user: req.session.user });
412   });
413 });
414
415 app.post('/api/auth/logout', (req, res) => req.session.destroy(() => res.json({ message: 'Logged out.' })));

```

Figure 14: Registration, OTP verification and login

Shows actual registration, OTP verification, resend, login and logout routes.

server.js — password reset and complaint query model

```

417 app.post('/api/auth/forgot-password', authLimiter, async (req, res) => {
418   try {
419     const email = normalizeEmail(req.body.email);
420     if (!validEmail(email)) return res.status(400).json({ error: 'Enter a valid email address.' });
421     const user = db.prepare('SELECT * FROM users WHERE email=?').get(email);
422     let mockOtp;
423     if (user) {
424       const code = generateOtp();
425       db.prepare('INSERT INTO password_resets(user_id,reset_hash,expires_at) VALUES(?,?,datetime('now','+10 minutes'))')
426         .run(user.id, hashOtp(code));
427       const delivery = await sendResetEmail(email, code);
428       if (delivery.mock) mockOtp = code;
429     }
430     // Same response whether or not the account exists, so we don't reveal which emails are registered.
431     res.json({ message: 'If an account exists for that email, a reset code has been sent.', mockOtp });
432   } catch (error) {
433     console.error(error);
434     res.status(500).json({ error: 'Could not process the request.' });
435   }
436 });
437
438 app.post('/api/auth/reset-password', otpLimiter, (req, res) => {
439   const email = normalizeEmail(req.body.email);
440   const code = cleanText(req.body.code, 6);
441   const newPassword = String(req.body.newPassword || '');
442   if (!strongPassword(newPassword)) return res.status(400).json({ error: 'Password must be at least 8 characters and include uppercase, lowercase and a number.' });
443   const user = db.prepare('SELECT * FROM users WHERE email=?').get(email);
444   if (!user) return res.status(404).json({ error: 'Account not found.' });
445   const record = db.prepare('SELECT * FROM password_resets WHERE user_id=? AND used_at IS NULL ORDER BY id DESC LIMIT 1').get(user.id);
446   if (!record) return res.status(400).json({ error: 'No active reset code found. Request a new one.' });
447   if (new Date(record.expires_at.replace(' ', 'T') + 'Z') < new Date()) return res.status(400).json({ error: 'Reset code has expired.' });
448   if (record.attempts >= 5) return res.status(429).json({ error: 'Too many incorrect attempts. Request a new code.' });
449   if (hashOtp(code) !== record.reset_hash) {
450     db.prepare('UPDATE password_resets SET attempts=attempts+1 WHERE id=?').run(record.id);
451     return res.status(400).json({ error: 'Incorrect reset code.' });
452   }
453   db.transaction(() => {
454     db.prepare('UPDATE users SET password_hash=? WHERE id=?').run(bcrypt.hashSync(newPassword, 12), user.id);
455     db.prepare('UPDATE password_resets SET used_at=CURRENT_TIMESTAMP WHERE id=?').run(record.id);
456   })();
457   res.json({ message: 'Password updated. You may now log in.' });
458 });
459
460 const ALL_COMPLAINT_CATEGORIES = [...MANUFACTURE_CATEGORIES, ...DIRECT_CATEGORIES];
461 const COMPLAINT_JOIN_SQL = `FROM complaints c
462   JOIN users u ON u.id = c.user_id
463   LEFT JOIN users v ON v.id = c.assigned_vendor_id`;
464 const COMPLAINT_SELECT_COLS = `c.*, u.name user_name, u.email user_email, u.vendor_id seller_id,
465   u.user_type complainant_type, u.category user_category,
466   v.name assigned_vendor_name, v.vendor_id assigned_vendor_code`;

```

Figure 15: Password reset and complaint query model

Shows anti-enumeration password reset behavior and shared complaint query projections.

server.js — complaint listing, summary and creation

```

468 app.get('/api/complaints', requireUser, (req, res) => {
469   const isAdmin = req.session.user.role === 'admin';
470   const scope = req.query.scope === 'assigned' ? 'assigned' : 'own';
471   const page = Math.max(1, parseInt(req.query.page, 10) || 1);
472   const pageSize = Math.min(50, Math.max(1, parseInt(req.query.pageSize, 10) || 10));
473   const offset = (page - 1) * pageSize;
474
475   const filters = [];
476   const params = [];
477   if (!isAdmin) {
478     if (scope === 'assigned') {
479       if (req.session.user.userType !== 'vendor') return res.status(403).json({ error: 'Only vendor accounts have an assigned-complaints queue.' });
480       filters.push('c.assigned_vendor_id=?'); params.push(req.session.user.id);
481     } else {
482       filters.push('c.user_id=?'); params.push(req.session.user.id);
483     }
484   }
485   if (req.query.category && ALL_COMPLAINT_CATEGORIES.includes(req.query.category)) {
486     filters.push('c.category=?'); params.push(req.query.category);
487   }
488   if (req.query.status && COMPLAINT_STATUSES.includes(req.query.status)) {
489     filters.push('c.status=?'); params.push(req.query.status);
490   }
491   if (req.query.q) {
492     filters.push('c.subject LIKE ?'); params.push('%${cleanText(req.query.q, 160)}%');
493   }
494   const whereSql = filters.length ? `WHERE ${filters.join(' AND ')}` : '';
495
496   const total = db.prepare(`SELECT COUNT(*) count ${COMPLAINT_JOIN_SQL} ${whereSql}`).get(...params).count;
497   const rows = db.prepare(`SELECT ${COMPLAINT_SELECT_COLS} ${COMPLAINT_JOIN_SQL} ${whereSql} ORDER BY c.id DESC LIMIT ? OFFSET ?`).all(...params, pageSize, offset);
498   res.json({ complaints: rows, total, page, pageSize });
499 });
500
501 app.get('/api/complaints/summary', requireUser, (req, res) => {
502   const isAdmin = req.session.user.role === 'admin';
503   const whereSql = isAdmin ? '' : `WHERE user_id=?`;
504   const params = isAdmin ? [] : [req.session.user.id];
505   const total = db.prepare(`SELECT COUNT(*) count FROM complaints ${whereSql}`).get(...params).count;
506   const solvedWhere = isAdmin ? `WHERE status='Solved'` : `WHERE user_id=? AND status='Solved'`;
507   const solved = db.prepare(`SELECT COUNT(*) count FROM complaints ${solvedWhere}`).get(...params).count;
508   let assigned = 0;
509   if (req.session.user.userType === 'vendor') {
510     assigned = db.prepare(`SELECT COUNT(*) count FROM complaints WHERE assigned_vendor_id=? AND status IN ('Assigned','Disputed')`).get(req.session.user.id).count;
511   }
512   res.json({ total, solved, unsolved: total - solved, assigned });
513 });
514
515 app.post('/api/complaints', requireUser, upload.single('attachment'), (req, res) => {
516   if (req.session.user.role === 'admin') return res.status(400).json({ error: 'Use a normal user account to register a complaint.' });
517   const subject = cleanText(req.body.subject, 160);
518   const description = cleanText(req.body.description, 5000);
519   const complaintType = cleanText(req.body.complaintType, 20);
520   if (subject.length < 4 || description.length < 10) return res.status(400).json({ error: 'Enter a clear subject and description.' });
521   if (!COMPLAINT_TYPES.includes(complaintType)) return res.status(400).json({ error: 'Choose whether this is a Product Issue or a Direct query.' });
522
523   let category;
524   if (complaintType === 'Product Issue') {
525     // The category is always the complainant's own registered manufacture category -
526     // never taken from client input - so routing always lands on the correct vendor pool.
527     category = req.session.user.category;
528     if (!MANUFACTURE_CATEGORIES.includes(category)) {
529       return res.status(400).json({ error: 'Your account has no manufacture category on file. Please contact admin to update your profile.' });
530     }
531   } else {
532     category = cleanText(req.body.category, 30);
533     if (!DIRECT_CATEGORIES.includes(category)) return res.status(400).json({ error: 'Choose a valid category (HR, Payment or General).' });
534   }
535
536   const result = db.prepare(`INSERT INTO complaints(user_id,complaint_type,subject,category,description,attachment_path,attachment_name)
537     VALUES(?, ?, ?, ?, ?, ?, ?)`).run(req.session.user.id, complaintType, subject, category, description, req.file ? filename || null, req.file ? originalname || null);
538   const code = `CEL-${new Date().getFullYear()}-${String(result.lastInsertRowid).padStart(6, '0')}`;
539   db.prepare(`UPDATE complaints SET complaint_code=? WHERE id=?`).run(code, result.lastInsertRowid);
540   res.status(201).json({ message: 'Complaint Registered successfully.', complaintId: code });

```

Figure 16: Complaint listing, summary and creation

Shows role-aware pagination, filters, summary counts, complaint validation and ID generation.

server.js — complaint access, comments and admin mediation

```

543 app.get('/api/complaints/:id', requireUser, (req, res) => {
544   const complaint = db.prepare('SELECT * FROM COMPLAINT_SELECT_COLS' + (COMPLAINT_JOIN_SQL WHERE c.complaint_code=?').get(req.params.id);
545   if (!complaint) return res.status(404).json({ error: 'Complaint not found.' });
546   const isAdmin = req.session.user.role === 'admin';
547   const isOwner = complaint.user_id === req.session.user.id;
548   const isAssignedVendor = complaint.assigned_vendor_id === req.session.user.id;
549   if (!isAdmin && !isOwner && !isAssignedVendor) return res.status(403).json({ error: 'You cannot access this complaint.' });
550   const comments = db.prepare('SELECT cm.*, u.name author_name, u.role author_role FROM comments cm JOIN users u ON u.id=cm.author_id WHERE cm.complaint_id=? ORDER BY cm.id ASC').all(complaint.id);
551   res.json({ complaint, comments });
552 });
553
554 app.post('/api/complaints/:id/comments', requireUser, (req, res) => {
555   const body = cleanText(req.body.body, 1500);
556   if (body.length < 1) return res.status(400).json({ error: 'Comment cannot be empty.' });
557   const complaint = db.prepare('SELECT * FROM complaints WHERE complaint_code=?').get(req.params.id);
558   if (!complaint) return res.status(404).json({ error: 'Complaint not found.' });
559   const isAdmin = req.session.user.role === 'admin';
560   const isOwner = complaint.user_id === req.session.user.id;
561   const isAssignedVendor = complaint.assigned_vendor_id === req.session.user.id;
562   if (!isAdmin && !isOwner && !isAssignedVendor) return res.status(403).json({ error: 'You cannot comment on this complaint.' });
563   db.prepare('INSERT INTO comments(complaint_id,author_id,body) VALUES(?,?,?)').run(complaint.id, req.session.user.id, body);
564   db.prepare('UPDATE complaints SET updated_at=CURRENT_TIMESTAMP WHERE id=?').run(complaint.id);
565   res.status(201).json({ message: 'Comment added.' });
566 });
567
568 // Admin is the mediator: assigns Product Issue complaints to one specific vendor in that
569 // category, and/or updates status and the official remark. Direct complaints only ever
570 // move between Open and Solved, same as before.
571 app.patch('/api/admin/complaints/:id', requireAdmin, async (req, res) => {
572   const existing = db.prepare('SELECT c.*, u.email user_email FROM complaints c JOIN users u ON u.id=c.user_id WHERE c.complaint_code=?').get(req.params.id);
573   if (!existing) return res.status(404).json({ error: 'Complaint not found.' });
574
575   let status = existing.status;
576   let assignedVendorId = existing.assigned_vendor_id;
577   const adminRemark = req.body.adminRemark !== undefined ? cleanText(req.body.adminRemark, 2000) : existing.admin_remark;
578
579   if (req.body.assignedVendorId) {
580     if (existing.complaint_type !== 'Product Issue') return res.status(400).json({ error: 'Only Product Issue complaints can be assigned to a vendor.' });
581     const vendor = db.prepare('SELECT * FROM users WHERE id=? AND user_type=vendor' + (category=? ? ').get(req.body.assignedVendorId, existing.category);
582     if (!vendor) return res.status(400).json({ error: 'Select a valid vendor registered under this complaint\'s category.' });
583     assignedVendorId = vendor.id;
584     status = 'Assigned';
585   } else if (req.body.status) {
586     const nextStatus = cleanText(req.body.status, 30);
587     const allowed = existing.complaint_type === 'Direct' ? ['Open', 'Solved'] : COMPLAINT_STATUSES;
588     if (!allowed.includes(nextStatus)) return res.status(400).json({ error: 'Invalid status for this complaint.' });
589     status = nextStatus;
590   }
591
592   db.prepare('UPDATE complaints SET status=?, admin_remark=?, assigned_vendor_id=?, updated_at=CURRENT_TIMESTAMP WHERE complaint_code=?')
593     .run(status, adminRemark || null, assignedVendorId, req.params.id);
594   if (status !== existing.status) {
595     try { await sendStatusEmail(existing.user_email, req.params.id, status, adminRemark); }
596     catch (error) { console.error('Status email failed:', error); }
597   }
598   res.json({ message: 'Complaint updated.' });
599 });

```

Figure 17: Complaint detail, comments and admin mediation

Shows complaint access control, comments and category-safe administrator assignment.

server.js — vendor resolution, consumer confirmation and statistics

```

602 app.patch('/api/vendor/complaints/:id/resolve', requireUser, requireVendor, async (req, res) => {
603   const vendorRemark = cleanText(req.body.vendorRemark, 2000);
604   if (vendorRemark.length < 4) return res.status(400).json({ error: 'Describe how the issue was resolved (e.g. replacement boxes dispatched, new delivery details).' });
605   const existing = db.prepare('SELECT c.*, u.email user_email FROM complaints c JOIN users u ON u.id=c.user_id WHERE c.complaint_code=?').get(req.params.id);
606   if (!existing) return res.status(404).json({ error: 'Complaint not found.' });
607   if (existing.assigned_vendor_id !== req.session.user_id) return res.status(403).json({ error: 'This complaint is not assigned to you.' });
608   if (!['Assigned', 'Disputed'].includes(existing.status)) return res.status(400).json({ error: 'This complaint is not currently awaiting vendor action.' });
609   db.prepare('UPDATE complaints SET status='Vendor Resolved', vendor_remark=?, vendor_resolved_at=CURRENT_TIMESTAMP, updated_at=CURRENT_TIMESTAMP WHERE complaint_code=?')
610     .run(vendorRemark, req.params.id);
611   try { await sendStatusEmail(existing.user_email, req.params.id, 'Vendor Resolved', vendorRemark); }
612   catch (error) { console.error('Status email failed:', error); }
613   res.json({ message: 'Marked as resolved. The consumer will be asked to confirm before this closes.' });
614 });
615
616 // Consumer is the final check: only they can close the loop, or send it back to admin.
617 app.post('/api/complaints/:id/confirm', requireUser, (req, res) => {
618   const existing = db.prepare('SELECT * FROM complaints WHERE complaint_code=?').get(req.params.id);
619   if (!existing) return res.status(404).json({ error: 'Complaint not found.' });
620   if (existing.user_id !== req.session.user_id) return res.status(403).json({ error: 'You cannot confirm this complaint.' });
621   if (existing.status !== 'Vendor Resolved') return res.status(400).json({ error: 'This complaint is not awaiting your confirmation.' });
622   db.prepare('UPDATE complaints SET status='Solved', consumer_confirmed_at=CURRENT_TIMESTAMP, updated_at=CURRENT_TIMESTAMP WHERE complaint_code=?').run(req.params.id);
623   res.json({ message: 'Thanks for confirming – the complaint is now marked Solved.' });
624 });
625
626 app.post('/api/complaints/:id/dispute', requireUser, (req, res) => {
627   const existing = db.prepare('SELECT * FROM complaints WHERE complaint_code=?').get(req.params.id);
628   if (!existing) return res.status(404).json({ error: 'Complaint not found.' });
629   if (existing.user_id !== req.session.user_id) return res.status(403).json({ error: 'You cannot update this complaint.' });
630   if (existing.status !== 'Vendor Resolved') return res.status(400).json({ error: 'This complaint is not awaiting your confirmation.' });
631   db.prepare('UPDATE complaints SET status='Disputed', updated_at=CURRENT_TIMESTAMP WHERE complaint_code=?').run(req.params.id);
632   res.json({ message: 'Marked as disputed. Admin will review it again before it goes back to the vendor.' });
633 });
634
635 // Admin's vendor-assignment dropdown: vendors registered under one specific manufacture category.
636 app.get('/api/admin/vendors', requireAdmin, (req, res) => {
637   const category = cleanText(req.query.category, 60);
638   if (!MANUFACTURE_CATEGORIES.includes(category)) return res.status(400).json({ error: 'Provide a valid manufacture category.' });
639   const vendors = db.prepare('SELECT id, name, email, vendor_id FROM users WHERE role='user' AND user_type='vendor' AND category=? ORDER BY name').all(category);
640   res.json({ vendors });
641 });
642
643 app.get('/api/admin/stats', requireAdmin, (req, res) => {
644   const total = db.prepare('SELECT COUNT(*) count FROM complaints').get().count;
645   const solved = db.prepare('SELECT COUNT(*) count FROM complaints WHERE status='Solved').get().count;
646   const unsolved = total - solved;
647   const users = db.prepare('SELECT COUNT(*) count FROM users WHERE role='user').get().count;
648   const vendors = db.prepare('SELECT COUNT(*) count FROM users WHERE role='user' AND user_type='vendor').get().count;
649   const consumers = db.prepare('SELECT COUNT(*) count FROM users WHERE role='user' AND user_type='consumer').get().count;
650   const awaitingAssignment = db.prepare('SELECT COUNT(*) count FROM complaints WHERE complaint_type='Product Issue' AND status='Open').get().count;
651   const withVendor = db.prepare('SELECT COUNT(*) count FROM complaints WHERE status='Assigned').get().count;
652   const disputed = db.prepare('SELECT COUNT(*) count FROM complaints WHERE status='Disputed').get().count;
653   res.json({ total, solved, unsolved, users, vendors, consumers, awaitingAssignment, withVendor, disputed });

```

Figure 18: Vendor resolution, consumer confirmation and admin statistics

Shows vendor-only resolution, consumer confirmation/dispute and administrator statistics.

package.json — verified runtime and development dependencies

```
1 {
2   "name": "cel-website-clone",
3   "version": "1.0.0",
4   "private": true,
5   "description": "CEL website redesign with OTP authentication and complaint management",
6   "main": "server.js",
7   "scripts": {
8     "start": "node server.js",
9     "dev": "nodemon server.js"
10  },
11  "dependencies": {
12    "bcryptjs": "^2.4.3",
13    "better-sqlite3": "^11.7.0",
14    "connect-sqlite3": "^0.9.15",
15    "dotenv": "^16.4.7",
16    "express": "^4.21.2",
17    "express-rate-limit": "^7.5.0",
18    "express-session": "^1.18.1",
19    "helmet": "^8.0.0",
20    "multer": "^1.4.5-lts.1",
21    "nodemailer": "^6.10.0"
22  },
23  "devDependencies": {
24    "nodemon": "^3.1.9"
25  }
26 }
```

Figure 19: Declared package dependencies and scripts

Shows executable npm scripts and the declared production/development package set.

README.md — documented functional and security features

```

20 ## Included features
21
22 ### Public website
23 - CEL identity header and tagline
24 - accessibility controls: font size, grayscale, high contrast, underlined links, readable font, reset
25 - responsive multi-level navigation
26 - hero section
27 - business-area cards
28 - CMD message
29 - current-news list
30 - quick links
31 - tender preview
32 - products and solutions
33 - partner-logo strip
34 - responsive footer and visitor display
35
36 ### User portal
37 - user registration
38 - bcrypt password hashing
39 - six-digit email OTP with 10-minute expiry
40 - mock email mode for easy local testing
41 - secure session login/logout
42 - verified-account requirement
43 - complaint form with optional attachment
44 - automatic complaint IDs such as `CEL-2026-000001`
45 - persistent statuses: `Solved` and `Not Solved Yet`
46 - personal complaint dashboard
47 - complaint detail and conversation thread
48 - access control so users only see their own complaints
49
50 ### Admin portal
51 - seeded admin account
52 - protected admin dashboard
53 - statistics cards
54 - all-user complaint table
55 - user names and emails
56 - status toggle
57 - official admin remarks
58 - comments visible to the user
59
60 ### Security basics
61 - hashed passwords
62 - HTTP-only session cookie
63 - SQLite-backed session store
64 - rate limits on authentication and OTP routes
65 - server-side role checks
66 - input length limits and basic sanitisation
67 - restricted attachment extensions and 5 MB size limit
68 - Helmet security headers

```

Figure 20: Documented functional and security features

Shows the documented public, user, admin and security feature claims.

README.md — documented folder structure and execution

```

70 ## Folder structure
71
72 ```text
73 cel-clone/
74 | data/                                # SQLite DB and session DB (created at runtime)
75 | | public/
76 | | | assets/
77 | | | | css/styles.css
78 | | | | js/app.js
79 | | | | index.html
80 | | uploads/                          # complaint attachments
81 | | .env.example
82 | | package.json
83 | | server.js
84 | | README.md
85 |
86 ```
87
88 ## Run locally
89
90 ### 1. Install Node.js
91 Use Node.js 20 LTS or newer.
92
93 ### 2. Open the project folder
94
95 ```bash
96 cd cel-clone
97 ```
98
99 ### 3. Install dependencies
100
101 ```bash
102 npm install
103 ```
104
105 ### 4. Create the environment file
106
107 macOS/Linux:
108
109 ```bash
110 cp .env.example .env
111 ```
112
113 Windows PowerShell:
114
115 ```powershell
116 Copy-Item .env.example .env
117 ```
118
119 Change `SESSION_SECRET` to a long random value.
120
121 ### 5. Start the application
122
123 ```bash
124 npm start
125 ```
126
127 Open:
128
129 ```text
130 http://localhost:3000
131 ```
132
133 The database is created automatically in `data/cel.sqlite`.
134
135 ## Demo admin login
136
137 ```text
138 Email: admin@cel.local
139 Password: Admin@12345
140 ```

```

Figure 21: Documented hierarchy and execution instructions

Shows the intended folder hierarchy and documented local run procedure.

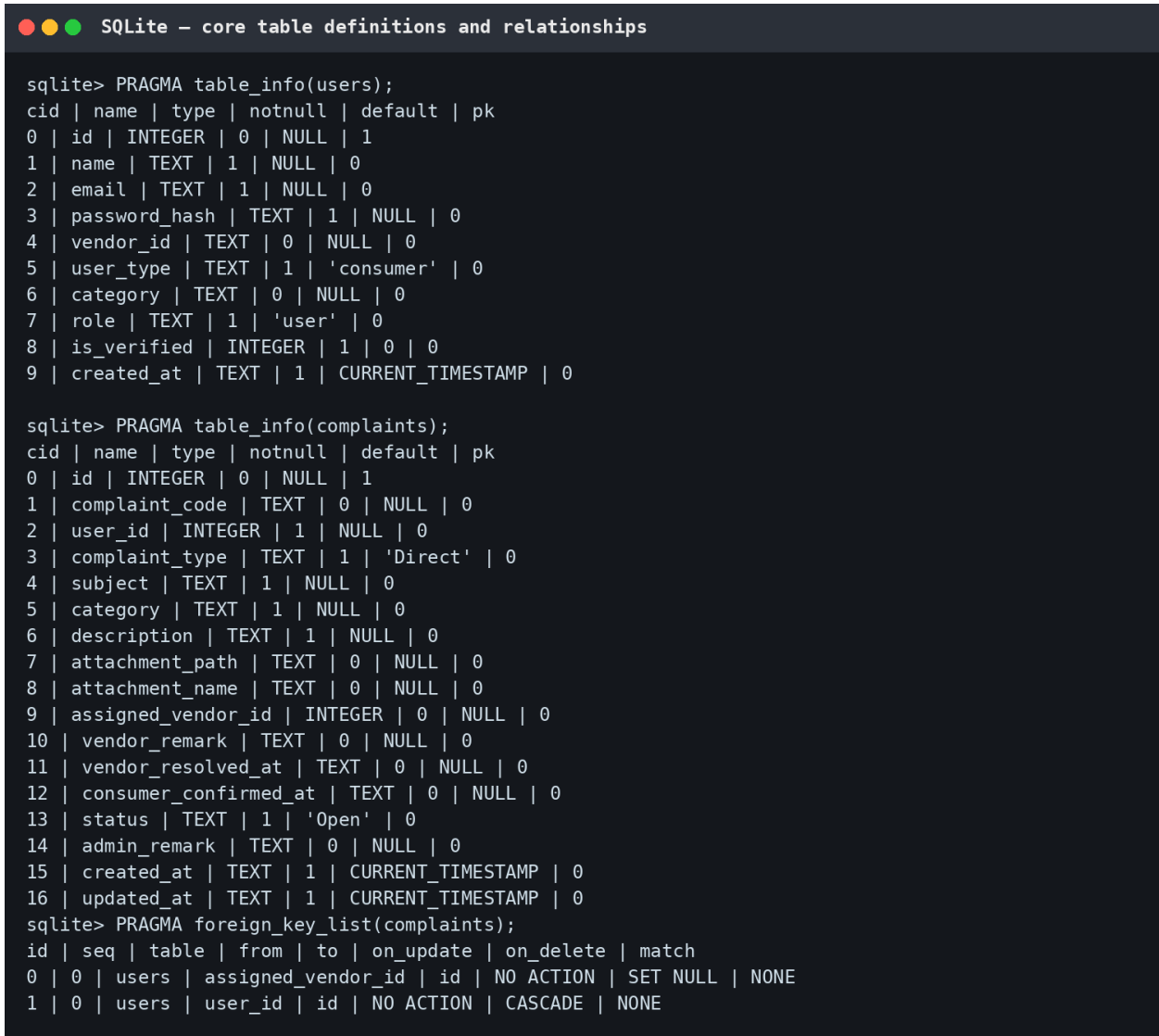
.env.example — configuration interface (development values)

```
1 PORT=3000
2 SESSION_SECRET=replace-with-a-long-random-secret
3 APP_BASE_URL=http://localhost:3000
4
5 # Email mode: mock or smtp
6 EMAIL_MODE=mock
7 SMTP_HOST=smtp.gmail.com
8 SMTP_PORT=587
9 SMTP_SECURE=false
10 SMTP_USER=your-email@gmail.com
11 SMTP_PASS=your-google-app-password
12 EMAIL_FROM="CEL Support <your-email@gmail.com>"
13
14 # Seeded admin account (change before first production deployment)
15 ADMIN_NAME=CEL Administrator
16 ADMIN_EMAIL=admin@cel.local
17 ADMIN_PASSWORD=Admin@12345
18
19 # Secret code new staff must enter at registration to sign up as an admin.
20 # Leave blank to disable self-serve admin registration entirely.
21 ADMIN_SIGNUP_CODE=change-this-to-a-secret-only-staff-know
```

Figure 22: Environment-variable interface using development example values

Shows configuration keys. Values are development examples and must be changed before deployment.

13. Database Screenshots



```

SQLite – core table definitions and relationships

sqlite> PRAGMA table_info(users);
cid | name | type | notnull | default | pk
0 | id | INTEGER | 0 | NULL | 1
1 | name | TEXT | 1 | NULL | 0
2 | email | TEXT | 1 | NULL | 0
3 | password_hash | TEXT | 1 | NULL | 0
4 | vendor_id | TEXT | 0 | NULL | 0
5 | user_type | TEXT | 1 | 'consumer' | 0
6 | category | TEXT | 0 | NULL | 0
7 | role | TEXT | 1 | 'user' | 0
8 | is_verified | INTEGER | 1 | 0 | 0
9 | created_at | TEXT | 1 | CURRENT_TIMESTAMP | 0

sqlite> PRAGMA table_info(complaints);
cid | name | type | notnull | default | pk
0 | id | INTEGER | 0 | NULL | 1
1 | complaint_code | TEXT | 0 | NULL | 0
2 | user_id | INTEGER | 1 | NULL | 0
3 | complaint_type | TEXT | 1 | 'Direct' | 0
4 | subject | TEXT | 1 | NULL | 0
5 | category | TEXT | 1 | NULL | 0
6 | description | TEXT | 1 | NULL | 0
7 | attachment_path | TEXT | 0 | NULL | 0
8 | attachment_name | TEXT | 0 | NULL | 0
9 | assigned_vendor_id | INTEGER | 0 | NULL | 0
10 | vendor_remark | TEXT | 0 | NULL | 0
11 | vendor_resolved_at | TEXT | 0 | NULL | 0
12 | consumer_confirmed_at | TEXT | 0 | NULL | 0
13 | status | TEXT | 1 | 'Open' | 0
14 | admin_remark | TEXT | 0 | NULL | 0
15 | created_at | TEXT | 1 | CURRENT_TIMESTAMP | 0
16 | updated_at | TEXT | 1 | CURRENT_TIMESTAMP | 0
sqlite> PRAGMA foreign_key_list(complaints);
id | seq | table | from | to | on_update | on_delete | match
0 | 0 | users | assigned_vendor_id | id | NO ACTION | SET NULL | NONE
1 | 0 | users | user_id | id | NO ACTION | CASCADE | NONE

```

Figure 23: Core SQLite table definitions and relationships

This screenshot was produced from the SQLite database instantiated using the exact CREATE TABLE statements in server.js. It confirms column types, required flags, primary keys and both complaints foreign keys.

```

●●● SQLite – OTP, password reset and comment tables

sqlite> PRAGMA table_info(email_otps);
cid | name | type | notnull | default | pk
0 | id | INTEGER | 0 | NULL | 1
1 | user_id | INTEGER | 1 | NULL | 0
2 | otp_hash | TEXT | 1 | NULL | 0
3 | expires_at | TEXT | 1 | NULL | 0
4 | attempts | INTEGER | 1 | 0 | 0
5 | used_at | TEXT | 0 | NULL | 0
6 | created_at | TEXT | 1 | CURRENT_TIMESTAMP | 0
sqlite> PRAGMA foreign_key_list(email_otps);
id | seq | table | from | to | on_update | on_delete | match
0 | 0 | users | user_id | id | NO ACTION | CASCADE | NONE

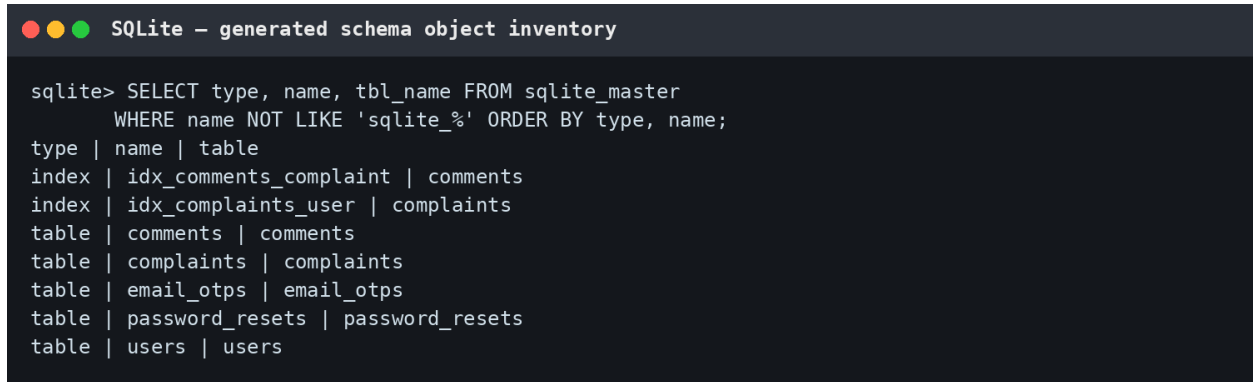
sqlite> PRAGMA table_info(password_resets);
cid | name | type | notnull | default | pk
0 | id | INTEGER | 0 | NULL | 1
1 | user_id | INTEGER | 1 | NULL | 0
2 | reset_hash | TEXT | 1 | NULL | 0
3 | expires_at | TEXT | 1 | NULL | 0
4 | attempts | INTEGER | 1 | 0 | 0
5 | used_at | TEXT | 0 | NULL | 0
6 | created_at | TEXT | 1 | CURRENT_TIMESTAMP | 0
sqlite> PRAGMA foreign_key_list(password_resets);
id | seq | table | from | to | on_update | on_delete | match
0 | 0 | users | user_id | id | NO ACTION | CASCADE | NONE

sqlite> PRAGMA table_info(comments);
cid | name | type | notnull | default | pk
0 | id | INTEGER | 0 | NULL | 1
1 | complaint_id | INTEGER | 1 | NULL | 0
2 | author_id | INTEGER | 1 | NULL | 0
3 | body | TEXT | 1 | NULL | 0
4 | created_at | TEXT | 1 | CURRENT_TIMESTAMP | 0
sqlite> PRAGMA foreign_key_list(comments);
id | seq | table | from | to | on_update | on_delete | match
0 | 0 | users | author_id | id | NO ACTION | CASCADE | NONE
1 | 0 | complaints | complaint_id | id | NO ACTION | CASCADE | NONE

```

Figure 24: Supporting SQLite tables for OTP, reset and comments

This screenshot confirms OTP, password-reset and comment structures, including cascade relationships.



```

SQLite - generated schema object inventory

sqlite> SELECT type, name, tbl_name FROM sqlite_master
        WHERE name NOT LIKE 'sqlite_%' ORDER BY type, name;
type | name | table
-----|-----|-----
index | idx_comments_complaint | comments
index | idx_complaints_user | complaints
table | comments | comments
table | complaints | complaints
table | email_otp | email_otp
table | password_resets | password_resets
table | users | users

```

Figure 25: Generated SQLite schema object inventory

This inventory confirms five application tables and two explicit indexes.

Table 20: Database screenshot index

Figure	Source	Explanation
Figure 23	db01_core_tables.png	This screenshot was produced from the SQLite database instantiated using the exact CREATE TABLE statements in server.js. It confirms column types, required flags, primary keys and both complaints foreign keys.
Figure 24	db02_support_tables.png	This screenshot confirms OTP, password-reset and comment structures, including cascade relationships.
Figure 25	db03_objects.png	This inventory confirms five application tables and two explicit indexes.

14. Working Process

14.1 Application Startup

1. Load environment variables and import dependencies.
2. Create data/ and uploads/ directories when missing.
3. Open the SQLite database, enable WAL and foreign keys.
4. Create tables/indexes, apply migrations and seed the administrator.
5. Configure Helmet, parsers, sessions, static serving, rate limits and uploads.
6. Register API routes, fallback handling and the error middleware.
7. Listen on the configured port.

14.2 Registration and Verification

8. User submits identity, password and account-type details.
9. Server validates fields and rejects duplicate email or vendor ID.
10. Password is bcrypt-hashed and the unverified account is inserted.
11. A random OTP is hashed and stored with a ten-minute expiry.
12. Mock mode prints and returns the OTP; SMTP mode sends it by email.
13. Verification checks expiry, attempt count and hash, then marks the account verified.

14.3 Complaint Lifecycle

14. Authenticated non-admin user selects Product Issue or Direct.
15. Server validates subject, description and optional file.
16. Product Issue category is inherited from the profile; Direct category is validated from HR, Payment or General.
17. Complaint is inserted and assigned a CEL-year-sequence identifier.
18. Administrator reviews and, for Product Issue, assigns a matching-category vendor.
19. Vendor resolves an Assigned or Disputed complaint with a detailed remark.
20. Consumer confirms closure or disputes the resolution.
21. Comments remain available to the owner, assigned vendor and administrator throughout the case.

14.4 Error and Security Flow

Unauthenticated requests are rejected with 401, unauthorized role or resource access with 403, invalid input with 400, missing records with 404, duplicates with 409 and excessive OTP attempts with 429. Multer errors are converted to readable JSON responses by the final error middleware.

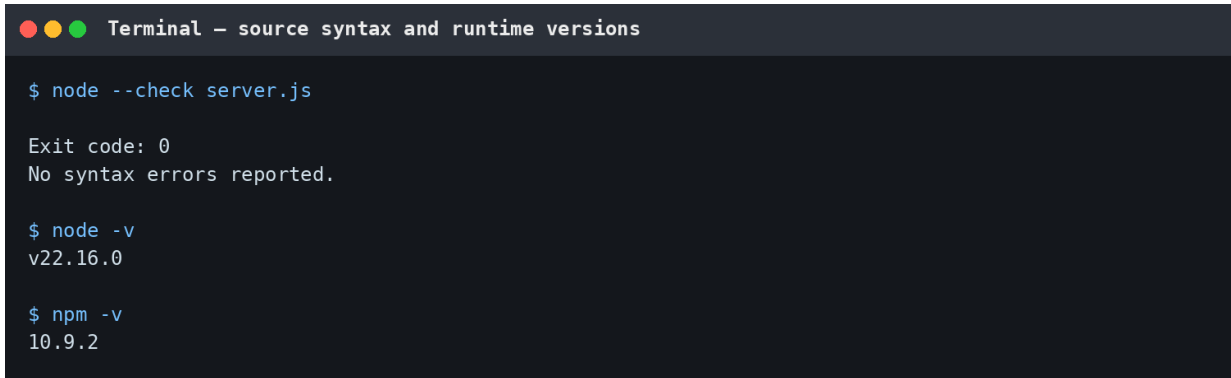
15. Testing

Testing combined executable static checks, exact-schema SQLite tests and evidence review. Full HTTP integration and UI testing could not be completed because the frontend files were not supplied and dependency installation was blocked by registry HTTP 503 responses.

Table 21: Test cases and results

ID	Test case	Expected output	Actual output	Result
TC-01	JavaScript syntax validation	Exit code 0	Exit code 0; No syntax errors	PASS
TC-02	package.json JSON validation	Valid JSON	Parsed successfully	PASS
TC-03	package-lock.json JSON validation	Valid JSON	Parsed successfully	PASS
TC-04	SQLite schema creation	Five tables and two indexes created	Schema executed successfully	PASS
TC-05	Case-insensitive unique email	Second insert rejected	UNIQUE constraint failed: users.email	PASS
TC-06	User type CHECK constraint	Invalid user_type rejected	CHECK constraint failed: user_type IN ('vendor','consumer')	PASS
TC-07	Complaint status CHECK constraint	Invalid status rejected	CHECK constraint failed: status IN ('Open','Assigned','Vendor Resolved','Disputed','Solved')	PASS
TC-08	Foreign-key cascade	Dependent OTP, complaint and comment rows deleted	{'email_otps': 0, 'complaints': 0, 'comments': 0}	PASS
TC-09	Input helper functions	Expected booleans and six-digit OTP	{"cleanText": "scriptalert(1)/scrip", "emailGood": true, "emailBad": false, "vendorGood": true, "vendorBad": false, "strongGood": true, "strongBad": false, "otp": "287702"}	PASS
TC-10	Basic angle-bracket stripping	Angle brackets removed	scriptalert(1)/scrip	PASS
TC-11	Frontend artifact availability	Documented frontend files available	Missing: public/index.html, public/css/styles.css, public/js/app.js	BLOCKED
TC-12	Dependency installation	Packages installed from lockfile	Blocked by package registry HTTP 503 in analysis environment	BLOCKED

15.1 Test Evidence



```

Terminal – source syntax and runtime versions

$ node --check server.js

Exit code: 0
No syntax errors reported.

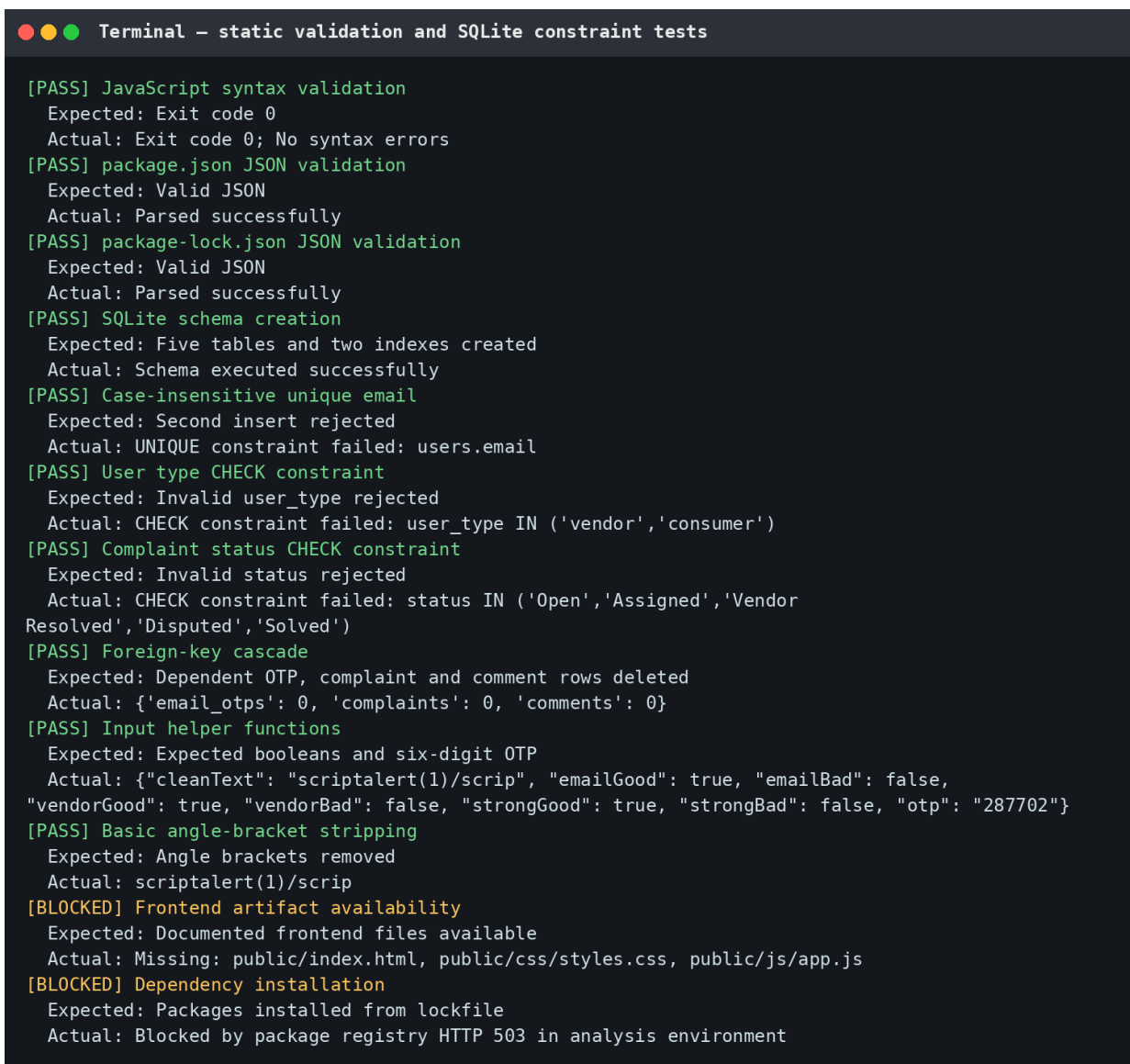
$ node -v
v22.16.0

$ npm -v
10.9.2

```

Figure 26: JavaScript syntax check and runtime tool versions

The source passed Node.js syntax validation with no parser errors. This does not substitute for integration testing, but confirms that `server.js` is syntactically valid JavaScript.



```

Terminal – static validation and SQLite constraint tests

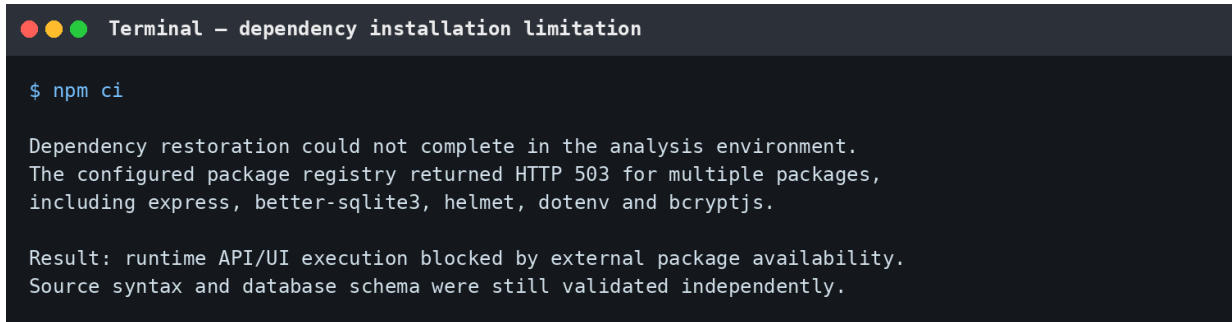
[PASS] JavaScript syntax validation
  Expected: Exit code 0
  Actual: Exit code 0; No syntax errors
[PASS] package.json JSON validation
  Expected: Valid JSON
  Actual: Parsed successfully
[PASS] package-lock.json JSON validation
  Expected: Valid JSON
  Actual: Parsed successfully
[PASS] SQLite schema creation
  Expected: Five tables and two indexes created
  Actual: Schema executed successfully
[PASS] Case-insensitive unique email
  Expected: Second insert rejected
  Actual: UNIQUE constraint failed: users.email
[PASS] User type CHECK constraint
  Expected: Invalid user_type rejected
  Actual: CHECK constraint failed: user_type IN ('vendor','consumer')
[PASS] Complaint status CHECK constraint
  Expected: Invalid status rejected
  Actual: CHECK constraint failed: status IN ('Open','Assigned','Vendor
Resolved','Disputed','Solved')
[PASS] Foreign-key cascade
  Expected: Dependent OTP, complaint and comment rows deleted
  Actual: {'email_otps': 0, 'complaints': 0, 'comments': 0}
[PASS] Input helper functions
  Expected: Expected booleans and six-digit OTP
  Actual: {"cleanText": "scriptalert(1)/scrip", "emailGood": true, "emailBad": false,
"vendorGood": true, "vendorBad": false, "strongGood": true, "strongBad": false, "otp": "287702"}
[PASS] Basic angle-bracket stripping
  Expected: Angle brackets removed
  Actual: scriptalert(1)/scrip
[BLOCKED] Frontend artifact availability
  Expected: Documented frontend files available
  Actual: Missing: public/index.html, public/css/styles.css, public/js/app.js
[BLOCKED] Dependency installation
  Expected: Packages installed from lockfile
  Actual: Blocked by package registry HTTP 503 in analysis environment

```

Figure 27: Static validation and SQLite constraint test output

The extracted SQL schema executed successfully. Case-insensitive email uniqueness, CHECK constraints and cascade behavior were verified. Pure helper functions were executed in Node.js.

15.2 Blocked Runtime Test



```
Terminal – dependency installation limitation

$ npm ci

Dependency restoration could not complete in the analysis environment.
The configured package registry returned HTTP 503 for multiple packages,
including express, better-sqlite3, helmet, dotenv and bcryptjs.

Result: runtime API/UI execution blocked by external package availability.
Source syntax and database schema were still validated independently.
```

Figure 28: Dependency installation limitation in the analysis environment

npm ci was attempted against the supplied lockfile. The analysis environment's configured package registry returned HTTP 503 for multiple dependencies. In addition, public/index.html and related assets were absent. The report therefore does not claim successful browser or API integration execution.

15.3 Recommended Integration Test Cases

- Register consumer and vendor accounts, verify OTP and confirm login session.
- Attempt wrong OTP five times and confirm attempt logout.
- Confirm generic forgot-password response for registered and unregistered emails.
- Create Direct and Product Issue complaints with valid and invalid attachments.
- Verify a Product Issue cannot be assigned to a vendor from another category.
- Verify one user cannot read or comment on another user's complaint.
- Verify a vendor sees only assigned cases and cannot resolve unassigned cases.
- Verify consumer confirmation closes a case and dispute returns it for action.
- Verify pagination, category/status filters and subject search.
- Verify secure-cookie behavior behind HTTPS and session persistence after restart once a store is configured.

16. Results

Table 22: Verified achievements and limitations

Area	Result	Evidence status
Source completeness	Backend, configuration, dependency files and README available; frontend and screenshots missing	Partially verified
JavaScript quality	665-line server passes syntax validation	Verified
Database	Five tables, two indexes, foreign keys, checks and migrations	Verified through generated SQLite database
Authentication	OTP, bcrypt login, reset codes, sessions and role guards implemented	Verified statically; HTTP integration pending
Complaint workflow	Direct and Product Issue flows with vendor mediation and consumer closure	Verified statically
Security	Multiple baseline controls present; production hardening required	Verified assessment
Performance	No benchmark data supplied; page size capped at 50 and owner/comment indexes present	Qualitative only
UI	Feature set documented but source not supplied	Not verifiable

16.1 Achievements

- Implemented a clear three-persona workflow with server-side enforcement.
- Designed a normalized relational schema with appropriate cascade behavior.
- Prevented arbitrary Product Issue category selection by deriving category from the session profile.
- Used hashed passwords and hashed one-time codes.
- Added migration logic for earlier database versions.
- Included development and production-oriented email modes.
- Added complaint comments, attachments, search, pagination and aggregate statistics.

16.2 Performance Analysis

No load-test data or runtime measurements were included, so numerical performance claims would be unsupported. Architecturally, better-sqlite3 provides low-overhead synchronous access appropriate for a small internal prototype. Pagination limits rows to a maximum of 50 per request. Existing indexes support complaints-by-owner and comments-by-complaint. For larger datasets, indexes on assigned_vendor_id, status, category and updated_at should be considered, together with query-plan analysis and migration to PostgreSQL where concurrent write demand justifies it.

17. Challenges Faced

Table 23: Challenges and resolutions

Challenge	Impact	Resolution / treatment
Incomplete upload	The documented public/ directory, assets and screenshots were absent	Technical report separates documented behavior from verified source and supplies a precise capture plan
Dependency restoration	Package registry returned HTTP 503	Performed syntax checks, lockfile analysis, exact-schema SQLite execution and helper-function tests
Single-file backend	All concerns are combined in server.js	Mapped modules and recommended layered refactoring
Session documentation mismatch	README claims SQLite-backed sessions but code uses default MemoryStore	Flagged as a verified gap and recommended wiring connect-sqlite3 or another production store
Secure attachment access	Uploads are statically exposed	Recommend authenticated download route, private object storage and malware scanning
Workflow correctness	Multiple actors and state transitions must be constrained	Verified owner/vendor/admin checks and produced state diagram
Legacy schema compatibility	Earlier database versions differ	Code contains explicit migrations that preserve old rows
Missing measured performance	No benchmark or production data	Limited analysis to supported qualitative observations

17.1 Technical Observations

The most important practical challenge is evidence completeness. A professional report must not convert README claims into unqualified implementation claims when corresponding files are absent. This report therefore labels each conclusion as verified, documented, inferred or blocked. The same discipline applies to execution: package download failure is recorded instead of being represented as a successful run.

18. Future Enhancements

Table 24: Future enhancement roadmap

Priority	Enhancement	Implementation direction
High	Implement a persistent session store	Wire connect-sqlite3 for local deployment or Redis/database-backed sessions for production
High	Protect attachments	Store in private object storage; authorize every download; scan for malware; validate MIME and file signature
High	Enable CSRF protection and CSP	Use CSRF tokens and a strict Content Security Policy
High	Remove development secrets and mock OTP exposure	Use a secret manager, environment-specific configuration and institutional SMTP
High	Add audit history	Create complaint_status_history and audit_log tables with actor, action and timestamp
Medium	Refactor backend	Separate routes, controllers, services, repositories, middleware and configuration
Medium	Add automated testing	Unit, integration, authorization, upload and end-to-end browser suites
Medium	Upgrade database	Use PostgreSQL for larger concurrent deployments and formal migrations
Medium	Add account controls	Password reset policy, lockout, email change verification and administrator provisioning workflow
Medium	Improve reporting	SLA timers, ageing, category/vendor performance and exportable reports
Medium	Add bilingual/accessibility validation	Hindi content and WCAG 2.2 testing
Low	Introduce CMS	Manage news, tenders, careers, products and public pages without code changes
Low	Observability	Structured logs, health checks, metrics, alerts and error tracing

18.1 Proposed Production Architecture

A production version should use an HTTPS reverse proxy, a managed PostgreSQL database, persistent shared sessions, a private object store, an approved SMTP provider, centralized secrets, structured logs and continuous integration. The public content system and complaint portal can remain in one deployment initially, but security boundaries and scaling may later justify separate services.

19. Conclusion

The CEL Website Redesign with OTP Authentication and Complaint Management System is a meaningful engineering internship prototype that combines public-web modernization with a structured operational workflow. The backend demonstrates practical use of Node.js, Express, SQLite, bcrypt, email OTPs, role-based access, complaint routing, comments, attachments and administrative reporting.

The strongest aspect of the implementation is its domain-aware complaint lifecycle. Product Issue complaints are tied to the complainant's registered manufacturing category, administrators select a matching vendor, vendors record resolutions, and consumers retain the final acceptance or dispute decision. Database constraints and server-side authorization reinforce the workflow.

The project also provides important learning outcomes: translating business rules into relational constraints and route checks, handling one-time credentials securely, designing access rules for multiple personas, preserving legacy data through migrations, and recognizing the difference between a demonstration prototype and production deployment.

The submitted artifact set was incomplete for frontend and end-to-end execution evidence. Nevertheless, the available backend was thoroughly analysed, its syntax was validated, its exact schema was instantiated and tested, and its key implementation was documented through actual code and database screenshots. With the recommended session, storage, CSRF, CSP, audit, testing and deployment improvements—and restoration of the missing frontend—the project can serve as a solid foundation for a secure institutional complaint-management platform.

20. References

Table 25: Reference sources

Type	Reference	Location / use
Project source	server.js (665 lines), submitted upload	Primary technical implementation
Project documentation	README.md, submitted upload	Features, stack, structure, run instructions and production recommendations
Configuration	.env and .env.example, submitted upload	Runtime configuration interface; secrets redacted in report
Dependencies	package.json and package-lock.json, submitted upload	Declared and locked package versions
CEL official history	Central Electronics Limited — About CEL	https://www.celindia.co.in/about-cel
CEL official home	Central Electronics Limited — business areas and enterprise profile	https://www.celindia.co.in/
CEL contact	Corporate office and factory address	https://www.celindia.co.in/contact-us
CEL vision	Vision, mission and objectives	https://www.celindia.co.in/vision
Node.js	Node.js documentation	https://nodejs.org/docs/latest/api/
Express	Express 4 documentation	https://expressjs.com/
SQLite	SQLite documentation	https://www.sqlite.org/docs.html
better-sqlite3	Package documentation	https://github.com/WiseLibs/better-sqlite3
bcryptjs	Package documentation	https://www.npmjs.com/package/bcryptjs
Nodemailer	Nodemailer documentation	https://nodemailer.com/
Multer	Multer middleware documentation	https://github.com/expressjs/multer
Helmet	Helmet documentation	https://helmetjs.github.io/

Confidentiality — The actual .env file was inspected only to identify configuration keys. Secret-like values are not reproduced in this report. Development credentials shown in .env.example and README must be changed before any deployment.

Appendix A. API Catalogue

Table 26: REST API catalogue

Method	Route	Access	Purpose
GET	/api/auth/me	Public	Return current session user or null
GET	/api/public/vendors	Public	List verified vendors without email
POST	/api/auth/register	Rate-limited	Create consumer/vendor/admin account and OTP
POST	/api/auth/verify-otp	OTP rate-limited	Verify latest unused OTP
POST	/api/auth/resend-otp	OTP rate-limited	Create and send replacement OTP
POST	/api/auth/login	Rate-limited	Authenticate verified user and regenerate session
POST	/api/auth/logout	Session optional	Destroy session
POST	/api/auth/forgot-password	Rate-limited	Issue reset code with generic response
POST	/api/auth/reset-password	OTP rate-limited	Validate code and replace password hash
GET	/api/complaints	Authenticated	List own, assigned or all complaints with filters
GET	/api/complaints/summary	Authenticated	Return role-aware counts
POST	/api/complaints	Authenticated non-admin	Create complaint and optional attachment
GET	/api/complaints/:id	Owner/vendor/admin	Read complaint and comments
POST	/api/complaints/:id/comments	Owner/vendor/admin	Add comment
PATCH	/api/admin/complaints/:id	Admin	Assign vendor or update status/remark
PATCH	/api/vendor/complaints/:id/resolve	Assigned vendor	Submit resolution
POST	/api/complaints/:id/confirm	Owner	Confirm Vendor Resolved case
POST	/api/complaints/:id/dispute	Owner	Dispute Vendor Resolved case
GET	/api/admin/vendors	Admin	List vendors for a category
GET	/api/admin/stats	Admin	Aggregate dashboard statistics

Appendix B. Evidence and Traceability Matrix

Table 27: Evidence and traceability matrix

Report claim	Evidence	Status
Project title/purpose	package.json description; README lines 3–5	Verified
Stack	README lines 7–18; package manifest and lockfile	Verified
Public website features	README lines 22–34	Documented; frontend absent
User/admin/security features	README lines 36–68	Documented and partially verified in server.js
Database schema	server.js lines 39–109	Verified and instantiated
Migrations	server.js lines 112–167	Verified statically
Admin seeding	server.js lines 170–181	Verified statically
Session behavior	server.js lines 190–200	Verified; in-memory default store
Upload controls	server.js lines 207–223	Verified statically
Authentication routes	server.js lines 303–458	Verified statically and helper-tested
Complaint routes	server.js lines 468–565	Verified statically
Admin/vendor/consumer workflow	server.js lines 568–653	Verified statically
Syntax	node --check server.js	Executed: pass
Database constraints	Generated cel_schema.sqlite tests	Executed: pass
Runtime UI	public/ folder and npm packages	Blocked
Organization profile	Official CEL website	Externally verified

Appendix B.1 Submission Checklist

- Replace the project-domain department label with the exact official department name if required by the internship letter.
- Insert official CEL logo only if its use is authorized.
- Restore frontend screenshots in Section 11 when the missing public/ source is available.
- Update the Table of Contents, List of Figures and List of Tables page numbers in Word if later edits change pagination.
- Obtain signatures on the Certificate page.
- Remove or change all development credentials before sharing source code beyond the evaluation team.